FACULTAD DE INGENIERIA
UNIVERSIDAD NACIONAL DE JUJUY

Herramientas Informáticas Avanzadas

Trabajo Practico Nº5
Optimizacion



Integrantes:

- *Fiorela Jimenez **LU:** APU006616*
- *Sebastián Velazque **LU:** APU006775*

Docentes:

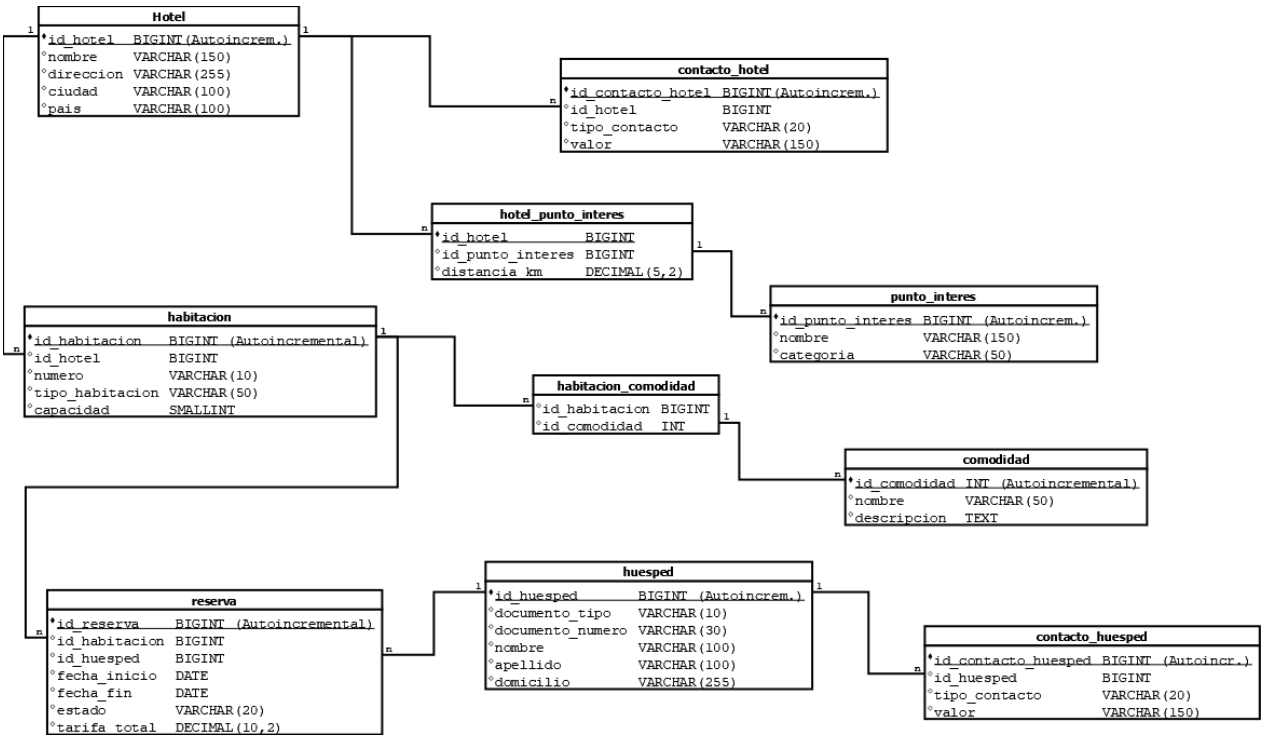
- *Prof. Alfredo Ronaldo Espinoza*
- *Prof. Angel Eduardo Casasola*

Desarrollo del TP de Optimización

1. Parte I — Analisis del Problema y Diseño de Datos

- **Hotel:** Información general, ubicación, medios de contacto (1 a N) y puntos de interés cercanos (N a M).
- **Habitación:** Pertenece a un hotel, tiene un tipo y un conjunto de comodidades (N a M).
- **Huésped:** Datos personales, domicilio y múltiples medios de contacto (teléfonos, correos) (1 a N).
- **Reserva:** Vincula un huésped con una habitación en un rango de fechas (fecha_inicio y fecha_fin). La tarifa total se congela al confirmar la reserva. El sistema contempla un control estricto de no solapamiento para evitar que una misma habitación sea reservada para períodos encimados.
- **Punto de Interés:** Nombre, categoría, distancia respecto al hotel.
- **Comodidad:** Descripción (WiFi, Aire Acondicionado, Jacuzzi, etc.).

Modelo relacional



Normalización vs Desnormalizacion

1. Tabla No Normalizada (Inicial)

id_hotel	nombre_hotel	nro_habitacion	comodidad	documento_huesped	nombre_huesped	fecha_inicio	fecha_fin	estado_reserva
1	Hotel Sol	101	WiFi	35123456	Juan Pérez	2026-10-15	2026-10-18	CONFIRMADA
1	Hotel Sol	101	TV	35123456	Juan Pérez	2026-10-15	2026-10-18	CONFIRMADA
1	Hotel Sol	102	WiFi	38987654	María Díaz	2026-10-20	2026-10-22	PENDIENTE

- **Primera Forma Normal (1FN):** En la estructura inicial no normalizada, información como las múltiples comodidades de una habitación o los distintos medios de contacto de un huésped se encontraban mezcladas en filas redundantes. Para cumplir con la 1FN, se asegura que todos los atributos contengan valores atómicos e indivisibles, desglosando los elementos múltiples en registros tabulares independientes.

1. Tabla No Normalizada (Inicial)

id_hotel	nombre_hotel	nro_habitacion	comodidad	documento_huesped	nombre_huesped	fecha_inicio	fecha_fin	estado_reserva
1	Hotel Sol	101	WiFi	35123456	Juan Pérez	2026-10-15	2026-10-18	CONFIRMADA
1	Hotel Sol	101	TV	35123456	Juan Pérez	2026-10-15	2026-10-18	CONFIRMADA
1	Hotel Sol	102	WiFi	38987654	María Díaz	2026-10-20	2026-10-22	PENDIENTE

- **Segunda Forma Normal (2FN):** Se eliminan las dependencias parciales asegurando que los datos dependan de toda la clave. Esto se evidencia en las tablas intermedias de resolución de relaciones muchos a muchos (N:M), como habitacion_comodidad y hotel_punto_interes, donde la clave primaria compuesta (id_habitacion + id_comodidad) sostiene de manera directa la relación, evitando duplicaciones innecesarias de atributos descriptivos.

1. Tabla No Normalizada (Inicial)

id_hotel	nombre_hotel	nro_habitacion	comodidad	documento_huesped	nombre_huesped	fecha_inicio	fecha_fin	estado_reserva
1	Hotel Sol	101	WiFi	35123456	Juan Pérez	2026-10-15	2026-10-18	CONFIRMADA
1	Hotel Sol	101	TV	35123456	Juan Pérez	2026-10-15	2026-10-18	CONFIRMADA
1	Hotel Sol	102	WiFi	38987654	María Díaz	2026-10-20	2026-10-22	PENDIENTE

2. Tabla: Hotel

id_hotel (PK)	nombre	direccion	ciudad	pais
1	Hotel Sol	Av. Belgrano 123	San Salvador de Jujuy	Argentina

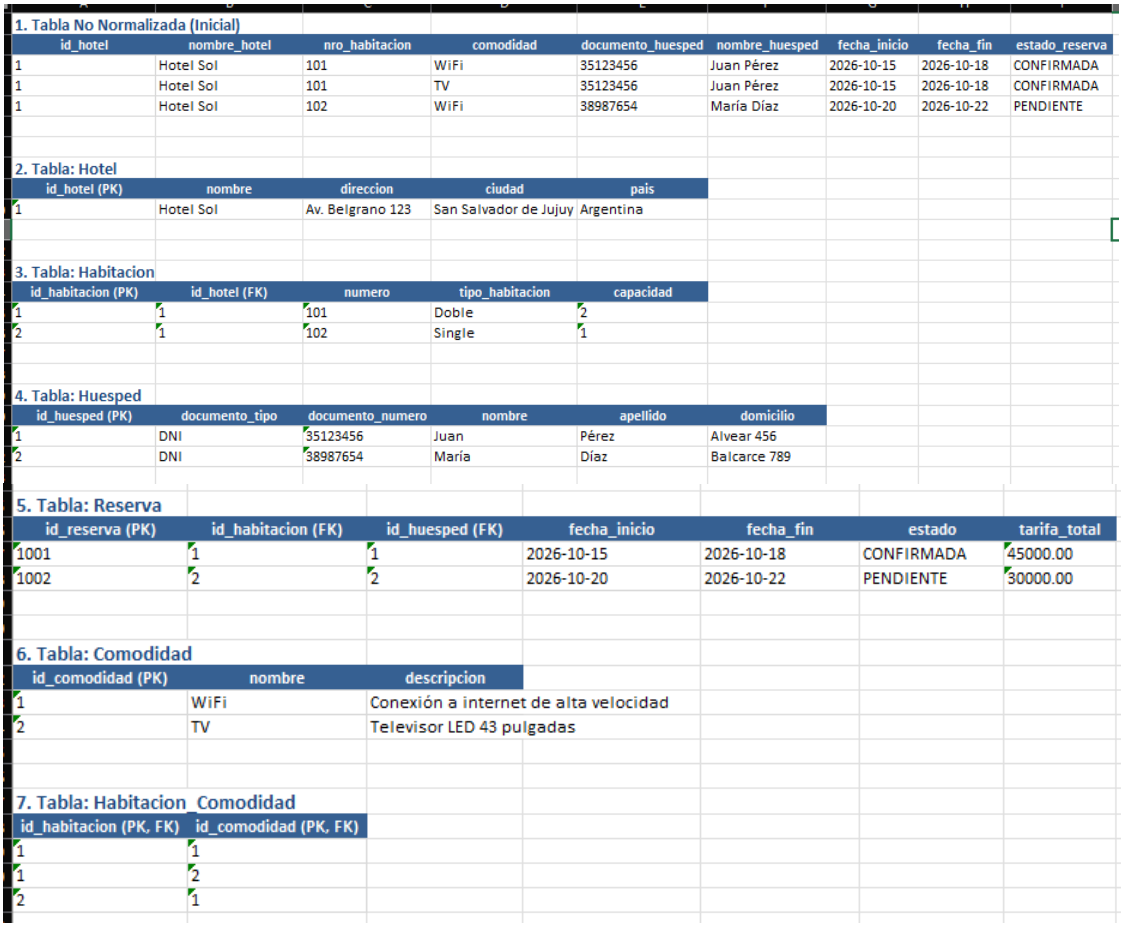
3. Tabla: Habitacion

id_habitacion (PK)	id_hotel (FK)	numero	tipo_habitacion	capacidad
1	1	101	Doble	2
2	1	102	Single	1

4. Tabla: Huesped

id_huesped (PK)	documento_tipo	documento_numero	nombre	apellido	domicilio
1	DNI	35123456	Juan	Pérez	Alvear 456
2	DNI	38987654	María	Díaz	Balcarce 789

- **Tercera Forma Normal (3FN):** Se eliminan las dependencias transitivas y la redundancia de datos mediante la separación estricta de las entidades del dominio. Por un lado, los datos generales del Hotel se administran de forma independiente y las habitaciones se vinculan únicamente mediante su clave foránea (id_hotel). Por otro lado, la entidad Reserva centraliza la relación transaccional entre el Huesped y la Habitacion en un rango de fechas determinado, evitando que la información personal de los clientes o los datos del hotel se repitan en cada operación.



1. Tabla No Normalizada (Inicial)

id_hotel	nombre_hotel	nro_habitacion	comodidad	documento_huesped	nombre_huesped	fecha_inicio	fecha_fin	estado_reserva
1	Hotel Sol	101	WiFi	35123456	Juan Pérez	2026-10-15	2026-10-18	CONFIRMADA
1	Hotel Sol	101	TV	35123456	Juan Pérez	2026-10-15	2026-10-18	CONFIRMADA
1	Hotel Sol	102	WiFi	38987654	María Díaz	2026-10-20	2026-10-22	PENDIENTE

2. Tabla: Hotel

id_hotel (PK)	nombre	direccion	ciudad	pais
1	Hotel Sol	Av. Belgrano 123	San Salvador de Jujuy	Argentina

3. Tabla: Habitacion

id_habitacion (PK)	id_hotel (FK)	numero	tipo_habitacion	capacidad
1	1	101	Doble	2
2	1	102	Single	1

4. Tabla: Huesped

id_huesped (PK)	documento_tipo	documento_numero	nombre	apellido	domicilio
1	DNI	35123456	Juan	Pérez	Alvear 456
2	DNI	38987654	María	Díaz	Balcarce 789

5. Tabla: Reserva

id_reserva (PK)	id_habitacion (FK)	id_huesped (FK)	fecha_inicio	fecha_fin	estado	tarifa_total
1001	1	1	2026-10-15	2026-10-18	CONFIRMADA	45000.00
1002	2	2	2026-10-20	2026-10-22	PENDIENTE	30000.00

6. Tabla: Comodidad

id_comodidad (PK)	nombre	descripcion
1	WiFi	Conexión a internet de alta velocidad
2	TV	Televisor LED 43 pulgadas

7. Tabla: Habitacion_Comodidad

id_habitacion (PK, FK)	id_comodidad (PK, FK)
1	1
1	2
2	1

Integridad y Reglas de Negocio

Claves Primarias (PK)

Elegimos claves primarias **subrogadas (autoincrementales)** para la mayoría de las tablas principales, ya que simplifican las relaciones, mejoran el rendimiento de los JOIN y evitan problemas si un dato del mundo real cambia:

- **hotel.id_hotel**: Identifica de forma única a cada hotel de la cadena.
- **habitacion.id_habitacion**: Identificador único por habitación. Preferimos usar este ID propio en lugar de depender únicamente del número de habitación, ya que el número se puede repetir entre diferentes hoteles.
- **huesped.id_huesped**: Identifica unívocamente a cada persona. Aunque el documento es único, usar un ID numérico propio simplifica las búsquedas e índices.
- **reserva.id_reserva**: Clave única para cada operación de reserva realizada en el sistema.
- **punto_interes.id_punto_interes** y **comodidad.id_comodidad**: Claves primarias para los catálogos globales de la cadena.
- **contacto_hotel.id_contacto_hotel** y **contacto_huesped.id_contacto_huesped**: Identifican cada medio de contacto individual.

Claves Primarias Compuestas

En las tablas intermedias que resuelven relaciones de muchos a muchos ($N:M$), utilizamos claves compuestas por las mismas claves foráneas que unen a las entidades:

- **hotel_punto_interes (id_hotel, id_punto_interes)**: La combinación de ambos evita que se registre el mismo punto de interés dos veces para un mismo hotel.
- **habitacion_comodidad (id_habitacion, id_comodidad)**: Garantiza que no se le asigne la misma comodidad dos veces a la misma habitación.

Claves Foráneas (FK) y Reglas de Integridad

Las claves foráneas nos permiten vincular la información de las tablas y mantener la integridad referencial del sistema:

- **habitacion.id_hotel** → **hotel.id_hotel**: Relaciona cada habitación con el hotel al que pertenece.
- **reserva.id_habitacion** → **habitacion.id_habitacion**: Asigna la habitación que se va a ocupar en la reserva.

- **reserva.id_huesped** → **huesped.id_huesped**: Asocia la reserva con el huésped correspondiente.
- **contacto_hotel.id_hotel** → **hotel.id_hotel** y **contacto_huesped.id_huesped** → **huesped.id_huesped**: Vinculan los múltiples teléfonos o e-mails con su respectivo dueño. Si se elimina un huésped o un hotel, estos contactos se eliminan en cascada (ON DELETE CASCADE) para no dejar información basura.

Restricciones de Unicidad (UNIQUE)

Además de las PKs, agregamos reglas para asegurar que ciertos datos no se dupliquen:

- **(documento_tipo, documento_numero) en huesped**: Evita cargar dos veces a la misma persona con el mismo documento.
- **(id_hotel, numero) en habitacion**: Permite que dos hoteles diferentes tengan la *"Habitación 101"*, pero no que el *mismo* hotel tenga dos habitaciones con el mismo número.

2. Diseño Parte II — Diseño Físico y Almacenamiento

Implementación física

Comando SQL

```
CREATE DATABASE hotel_db;

-- Tabla Hotel
CREATE TABLE hotel (
    id_hotel BIGSERIAL PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL,
    direccion VARCHAR(255) NOT NULL,
    ciudad VARCHAR(100) NOT NULL,
    pais VARCHAR(100) NOT NULL
);

-- Contacto Hotel
CREATE TABLE contacto_hotel (
    id_contacto_hotel BIGSERIAL PRIMARY KEY,
    id_hotel BIGINT NOT NULL,
    tipo_contacto VARCHAR(20) NOT NULL,
```

```
-- Punto de Interés
CREATE TABLE punto_interes (
    id_punto_interes BIGSERIAL PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL,
    categoria VARCHAR(50) NOT NULL
);

-- Hotel - Punto de Interés (N:M)
CREATE TABLE hotel_punto_interes (
    id_hotel BIGINT NOT NULL,
    id_punto_interes BIGINT NOT NULL,
    distancia_km DECIMAL(5,2) NOT NULL,
    PRIMARY KEY (id_hotel, id_punto_interes),
    CONSTRAINT fk_hp_hotel FOREIGN KEY (id_hotel) REFERENCES hotel(id_hotel) ON DELETE CASCADE,
    CONSTRAINT fk_hp_punto FOREIGN KEY (id_punto_interes) REFERENCES punto_interes(id_punto_interes) ON DELETE CASCADE
);
```

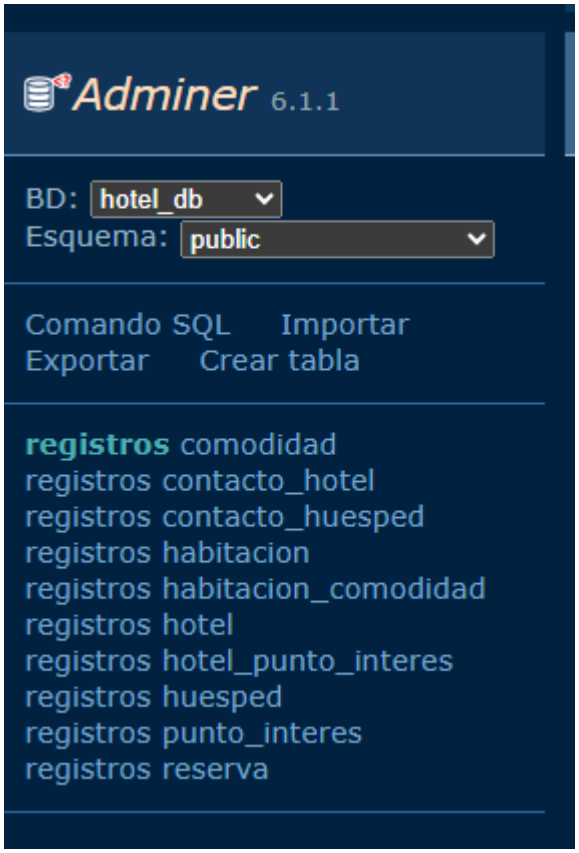
```sql
-- Habitación
CREATE TABLE habitacion (
    id_habitacion BIGSERIAL PRIMARY KEY,
    id_hotel BIGINT NOT NULL,
    numero VARCHAR(10) NOT NULL,
    tipo_habitacion VARCHAR(50) NOT NULL,
    capacidad SMALLINT NOT NULL,
    CONSTRAINT fk_habitacion_hotel FOREIGN KEY (id_hotel)
        REFERENCES hotel(id_hotel) ON DELETE RESTRICT,
    CONSTRAINT uq_hotel_numero UNIQUE (id_hotel, numero)
);

-- 6. Tabla Comodidad
CREATE TABLE comodidad (
    id_comodidad SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE,
    descripcion TEXT
);
```

```sql
-- Habitación - Comodidad (N:M)
CREATE TABLE habitacion_comodidad (
    id_habitacion BIGINT NOT NULL,
    id_comodidad INT NOT NULL,
    PRIMARY KEY (id_habitacion, id_comodidad),
    CONSTRAINT fk_hc_habitacion FOREIGN KEY (id_habitacion)
        REFERENCES habitacion(id_habitacion) ON DELETE CASCADE,
    CONSTRAINT fk_hc_comodidad FOREIGN KEY (id_comodidad)
        REFERENCES comodida(id_comodidad) ON DELETE CASCADE
);
-- Huésped
CREATE TABLE huesped (
    id_huesped BIGSERIAL PRIMARY KEY,
    documento_tipo VARCHAR(10) NOT NULL,
    documento_numero VARCHAR(30) NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    domicilio VARCHAR(255),
    CONSTRAINT uq_huesped_doc UNIQUE (documento_tipo, documento_numero)
);
```

```sql
-- Contacto Huésped
CREATE TABLE contacto_huesped (
    id_contacto_huesped BIGSERIAL PRIMARY KEY,
    id_huesped BIGINT NOT NULL,
    tipo_contacto VARCHAR(20) NOT NULL,
    valor VARCHAR(150) NOT NULL,
    CONSTRAINT fk_contacto_huesped FOREIGN KEY (id_huesped)
        REFERENCES huesped(id_huesped) ON DELETE CASCADE
);
```

```sql
-- Reserva
CREATE TABLE reserva (
    id_reserva BIGSERIAL PRIMARY KEY,
    id_habitacion BIGINT NOT NULL,
    id_huesped BIGINT NOT NULL,
    fecha_inicio DATE NOT NULL,
    fecha_fin DATE NOT NULL,
    estado VARCHAR(20) NOT NULL,
    tarifa_total DECIMAL(10,2) NOT NULL,
    CONSTRAINT fk_reserva_habitacion FOREIGN KEY (id_habitacion) REFERENCES habitacion(id_habitacion),
    CONSTRAINT fk_reserva_huesped FOREIGN KEY (id_huesped) REFERENCES huesped(id_huesped)
);
```



<u>Justificación de los tipos de datos seleccionados</u>

Para definir los tipos de datos de las tablas nos basamos principalmente en el volumen estimado que va a manejar el sistema (sabiendo que la tabla de reservas va a superar los 6 millones de registros) y en el tipo de información que guarda cada campo:

BIGINT vs INT (Elección de Claves Primarias):

- Para las tablas principales como `reserva`, `huesped` y `habitacion` elegimos usar `BIGINT` (`BIGSERIAL`). Como el trabajo exige cargar

más de 6 millones de reservas de prueba y en un entorno real con rotación constante este número crece rapidísimo, preferimos asegurar que no nos vayamos a quedar cortos con el límite de un entero normal (`INT` llega hasta 2 mil millones, pero con IDs autoincrementales y posibles borrados es mejor no arriesgar).
- En cambio, para tablas de catálogo chico como `comodidad`, dejamos un `INT` (`SERIAL`) común. Un hotel difícilmente tenga más de un par de decenas o cientos de comodidades (WiFi, AC, TV, etc.), así que no tenía sentido gastar 8 bytes por registro.

Tipos de Texto (VARCHAR y TEXT vs CHAR):

- Para nombres, direcciones, e-mails y documentos usamos `VARCHAR` definiendo un límite razonable según el campo. De esta manera el motor solo ocupa el espacio real que ocupa el texto y no desperdiciamos espacio.
- Para `comodidad.descripcion` usamos `TEXT` porque hay descripciones que pueden ser muy cortas y otras más detalladas.
- Descartamos usar `CHAR` en todos los campos de texto porque rellena con espacios en blanco hasta llegar al límite fijo, lo cual en tablas con millones de filas termina ocupando un montón de disco y memoria inútilmente.

Fechas (DATE vs TIMESTAMP):

- Para `fecha_inicio` y `fecha_fin` en las reservas elegimos `DATE` (que ocupa 3 bytes) en lugar de `TIMESTAMP` (8 bytes). La lógica del negocio de un hotel maneja las reservas por día (check-in y check-out a nivel fecha). No necesitábamos guardar la hora exacta con segundos, por lo que ahorramos un 60% de espacio en disco por cada fecha guardada en los 6 millones de filas.

Valores Numéricos (DECIMAL vs FLOAT):

- Para `tarifa_total` en `reserva` usamos `DECIMAL(10,2)`. Al tratarse de dinero no podemos usar `FLOAT` o `DOUBLE` porque el punto flotante introduce errores de redondeo al hacer sumas o cálculos. `DECIMAL` nos asegura precisión exacta hasta los centavos.

- Para `distancia_km` en los puntos de interés usamos `DECIMAL(5,2)`, que nos permite guardar distancias exactas de hasta 999.99 km con dos decimales.

Motor y Estructuras de Almacenamiento

A diferencia de otros motores como MySQL (que suelen usar InnoDB como un componente separado), PostgreSQL maneja todo el almacenamiento de forma interna y directa. Organiza la información en páginas o bloques fijos de 8 KB en el disco. Lo más copado que tiene es cómo maneja la concurrencia con algo llamado MVCC (Control de Concurrencia Multiversión): básicamente, cuando leés datos y otros escriben al mismo tiempo, no se traban ni se bloquean entre sí. Eso sí, cuando hacés updates o borrados masivos con millones de filas, el motor va dejando versiones viejas de los registros, por lo que usa un proceso automático de limpieza en segundo plano (el autovacuum) para que el disco no se infle de basura.

Estrategias de Almacenamiento y Entorno de Experimentación

Todo esto del almacenamiento se nota de verdad cuando empezás a exigir a la base de datos. Para probar el sistema, armé todo el entorno de pruebas levantando un contenedor de Docker con PostgreSQL, y para manejar las consultas y ver las tablas usé Adminer.

Hacer pruebas con una tabla pesada de más de 6 millones de registros en reserva te abre los ojos sobre por qué son tan importantes los índices. Cuando hacés una consulta sin índice, el motor se pone a hacer un Seq Scan (recorre las 6 millones de filas una por una leyendo todo el disco), lo que tarda varios segundos y te congela todo. En cambio, usando los índices B-Tree bien armados, el sistema salta directo al bloque exacto de datos y te devuelve el resultado en milisegundos.

3. Parte III — Índices y Estructuras de Acceso

Estrategia general de índices



tablename	indexname	indexdef
comodidad	comodidad_nombre_key	CREATE UNIQUE INDEX comodidad_nombre_key ON public.comodidad USING btree (nombre)
comodidad	comodidad_pkey	CREATE UNIQUE INDEX comodidad_pkey ON public.comodidad USING btree (id_comodidad)
contacto_hotel	contacto_hotel_pkey	CREATE UNIQUE INDEX contacto_hotel_pkey ON public.contacto_hotel USING btree (id_contacto_hotel)
contacto_hotel	idx_fk_contacto_hotel_hotel	CREATE INDEX idx_fk_contacto_hotel_hotel ON public.contacto_hotel USING btree (id_hotel)
contacto_huesped	contacto_huesped_pkey	CREATE UNIQUE INDEX contacto_huesped_pkey ON public.contacto_huesped USING btree (id_contacto_huesped)
contacto_huesped	idx_fk_contacto_huesped_huesped	CREATE INDEX idx_fk_contacto_huesped_huesped ON public.contacto_huesped USING btree (id_huesped)
habitacion	habitacion_pkey	CREATE UNIQUE INDEX habitacion_pkey ON public.habitacion USING btree (id_habitacion)
habitacion	idx_fk_habitacion_hotel	CREATE INDEX idx_fk_habitacion_hotel ON public.habitacion USING btree (id_hotel)
habitacion	uq_hotel_numero	CREATE UNIQUE INDEX uq_hotel_numero ON public.habitacion USING btree (id_hotel, numero)
habitacion_comodidad	habitacion_comodidad_pkey	CREATE UNIQUE INDEX habitacion_comodidad_pkey ON public.habitacion_comodidad USING btree (id_habitacion, id_comodidad)
hotel	hotel_pkey	CREATE UNIQUE INDEX hotel_pkey ON public.hotel USING btree (id_hotel)
hotel_punto_interes	hotel_punto_interes_pkey	CREATE UNIQUE INDEX hotel_punto_interes_pkey ON public.hotel_punto_interes USING btree (id_hotel, id_punto_interes)
huesped	huesped_pkey	CREATE UNIQUE INDEX huesped_pkey ON public.huesped USING btree (id_huesped)
huesped	idx_huesped_apellido	CREATE INDEX idx_huesped_apellido ON public.huesped USING btree (apellido)
huesped	uq_huesped_doc	CREATE UNIQUE INDEX uq_huesped_doc ON public.huesped USING btree (documento_tipo, documento_numero)
punto_interes	punto_interes_pkey	CREATE UNIQUE INDEX punto_interes_pkey ON public.punto_interes USING btree (id_punto_interes)
reserva	idx_fk_reserva_huesped	CREATE INDEX idx_fk_reserva_huesped ON public.reserva USING btree (id_huesped)
reserva	idx_reserva_estado	CREATE INDEX idx_reserva_estado ON public.reserva USING btree (estado)
reserva	idx_reserva_fechas	CREATE INDEX idx_reserva_fechas ON public.reserva USING btree (fecha_inicio, fecha_fin)
reserva	idx_reserva_habitacion_fechas	CREATE INDEX idx_reserva_habitacion_fechas ON public.reserva USING btree (id_habitacion, fecha_inicio, fecha_fin)
reserva	reserva_pkey	CREATE UNIQUE INDEX reserva_pkey ON public.reserva USING btree (id_reserva)

Justificación y Análisis Individual de Índices

1. Índice Compuesto: idx_reserva_habitacion_fechas

- **Columnas:** reserva(id_habitacion, fecha_inicio, fecha_fin)
- **Consultas que pretende mejorar:** Búsqueda de disponibilidad de una habitación específica dentro de un rango de fechas (WHERE id_habitacion = X AND fecha_inicio < Y AND fecha_fin > Z), validación de no solapamiento de reservas.
- **Justificación:** Es la consulta más crítica del sistema. Al ser un índice compuesto con id_habitacion como primer prefijo, el motor evalúa directamente el índice en un árbol B-Tree sin tener que realizar un escaneo secuencial (*Seq Scan*) sobre los 6+ millones de filas de reservas.

2. Índice Compuesto: idx_reserva_fechas

- **Columnas:** reserva(fecha_inicio, fecha_fin)
- **Consultas que pretende mejorar:** Reportes de ocupación global, listados de reservas activas en un rango de fechas a nivel de toda la cadena de hoteles.
- **Justificación:** Permite filtrar de forma directa por periodos temporales cuando no se filtra por una habitación en particular.

3. Índice sobre Clave Foránea: idx_fk_reserva_huesped

- **Columnas:** reserva(id_huesped)

- **Consultas que pretende mejorar:** INNER JOIN entre huesped y reserva, historial de reservas hechas por un cliente en particular (WHERE id_huesped = X).
- **Justificación:** A diferencia de otros motores, PostgreSQL no crea automáticamente índices sobre claves foráneas. Sin este índice, cualquier JOIN entre huéspedes y reservas forzaría un recorrido completo de la tabla de reservas.

4. Índice Simple: idx_huesped_apellido
- **Columnas:** huesped(apellido)
- **Consultas que pretende mejorar:** Búsquedas en la recepción o en el sistema por el apellido del cliente (WHERE apellido = 'Pérez').
- **Justificación:** Mejora la experiencia de uso en operaciones diarias donde no siempre se cuenta con el número exacto de documento del cliente.

5. Índice sobre Clave Foránea: idx_fk_habitacion_hotel
- **Columnas:** habitacion(id_hotel)
- **Consultas que pretende mejorar:** Listar todas las habitaciones de un hotel en particular, consultas agregadas de capacidad total por hotel.
- **Justificación:** Optimiza las operaciones de JOIN entre la tabla hotel y habitacion.

Funcionamiento del Índice B-Tree
En la captura de Adminer se observa que todos los índices figuran con la cláusula USING btree.
- **Estructura Interna:** El índice **B-Tree (Árbol Balanceado)** organiza las claves de manera jerárquica mediante tres niveles: **Nodo Raíz** (punto de entrada), **Nodos Internos** (guías de navegación) y **Nodos Hoja** (que contienen el valor indexado y el puntero físico TID hacia el registro en disco).
- **Eficiencia con 6+ Millones de Filas:** En una búsqueda sin índice (*Seq Scan*), PostgreSQL debe leer las 6,000,000 de filas del disco. Al utilizar B-Tree, la altura del árbol no supera los 3-4 niveles, lo que permite localizar cualquier reserva realizando únicamente **3 o 4 lecturas de bloques de disco (*Index Scan*)**, reduciendo el tiempo de respuesta de segundos a milisegundos.

Regla del Prefijo Izquierdo (Leftmost Prefix Rule)
Tomando como referencia el índice compuesto de la captura:

```
CREATE INDEX idx_reserva_habitacion_fechas
ON public.reserva USING btree (id_habitacion, fecha_inicio, fecha_fin)
```

La **Regla del Prefijo Izquierdo** establece que el motor de base de datos solo utilizará este índice compuesto si la consulta filtra obligatoriamente por el **primer campo definido más a la izquierda** (id_habitacion).

Justificación con Ejemplos de Consultas:

- **Caso A (Sigue la regla — Aprovecha el Índice):**

```
SELECT * FROM reserva
WHERE id_habitacion = 10
  AND fecha_inicio >= '2026-10-01';
```

Resultado: Realiza un **Index Scan**. Al incluir id_habitacion, el motor entra directamente por la rama correspondiente en el B-Tree y filtra las fechas dentro de esa habitación.

- **Caso B (Viola la regla — Descarta el Índice):**

```
SELECT * FROM reserva
WHERE fecha_inicio = '2026-10-01';
```

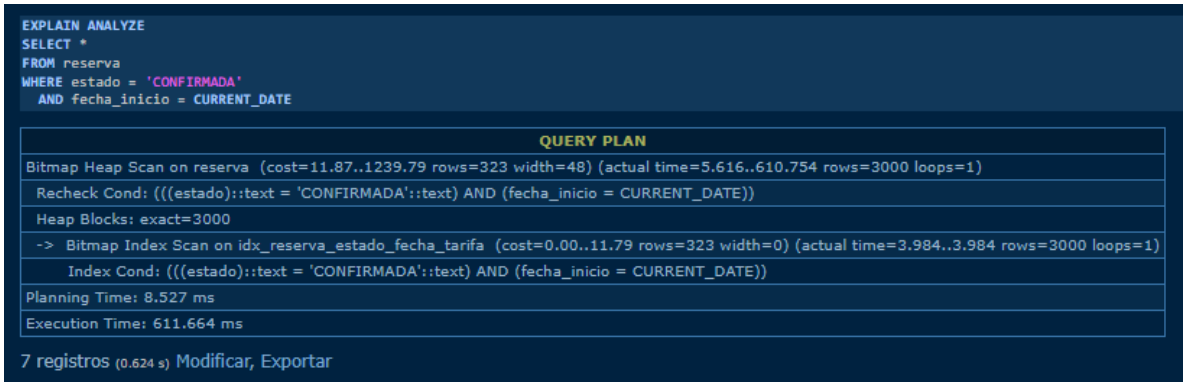
Resultado: Realiza un **Seq Scan** (recorrido completo de las 6M de filas). Al omitir id_habitacion, las fechas están dispersas en distintas ramas del árbol y el índice compuesto no resulta útil.

Escenario A: Consulta con la Primera Columna (Prefijo Izquierdo Exacto)



Resultado esperado: Utiliza el índice (Bitmap Index Scan / Index Scan). Al incluir la columna inicial del índice, el motor puede navegar el árbol B-Tree.

Escenario B: Consulta con las Dos Primeras Columnas en Orden (Prefijo Contiguo)



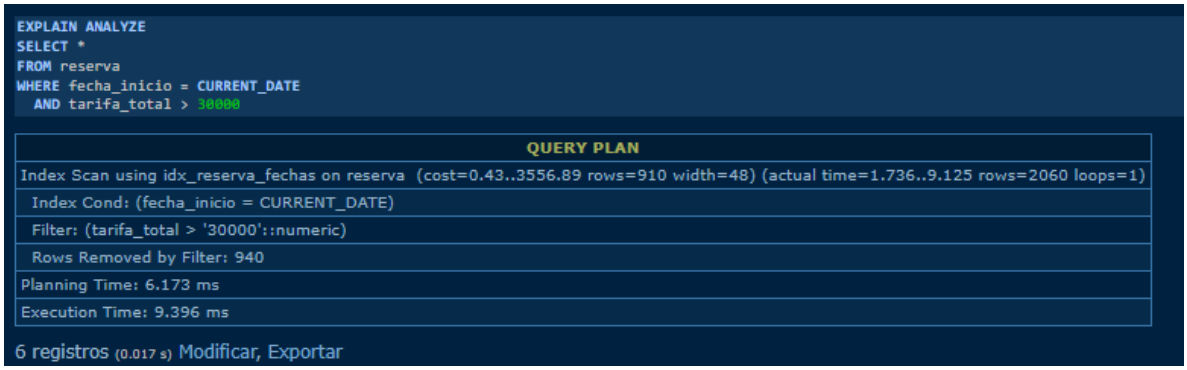
```
EXPLAIN ANALYZE
SELECT *
FROM reserva
WHERE estado = 'CONFIRMADA'
  AND fecha_inicio = CURRENT_DATE
```

QUERY PLAN
Bitmap Heap Scan on reserva (cost=11.87..1239.79 rows=323 width=48) (actual time=5.616..610.754 rows=3000 loops=1)
Recheck Cond: (((estado)::text = 'CONFIRMADA'::text) AND (fecha_inicio = CURRENT_DATE))
Heap Blocks: exact=3000
-> Bitmap Index Scan on idx_reserva_estado_fecha_tarifa (cost=0.00..11.79 rows=323 width=0) (actual time=3.984..3.984 rows=3000 loops=1)
Index Cond: (((estado)::text = 'CONFIRMADA'::text) AND (fecha_inicio = CURRENT_DATE))
Planning Time: 8.527 ms
Execution Time: 611.664 ms

7 registros (0.624 s) Modificar, Exportar

Resultado esperado: Utiliza el índice con máxima eficiencia. Filtra rápidamente por el primer nivel del árbol (estado) y reduce de inmediato las ramas según la fecha.

Escenario C: Consulta Omitiendo la Primera Columna (Rompe el Prefijo Izquierdo)



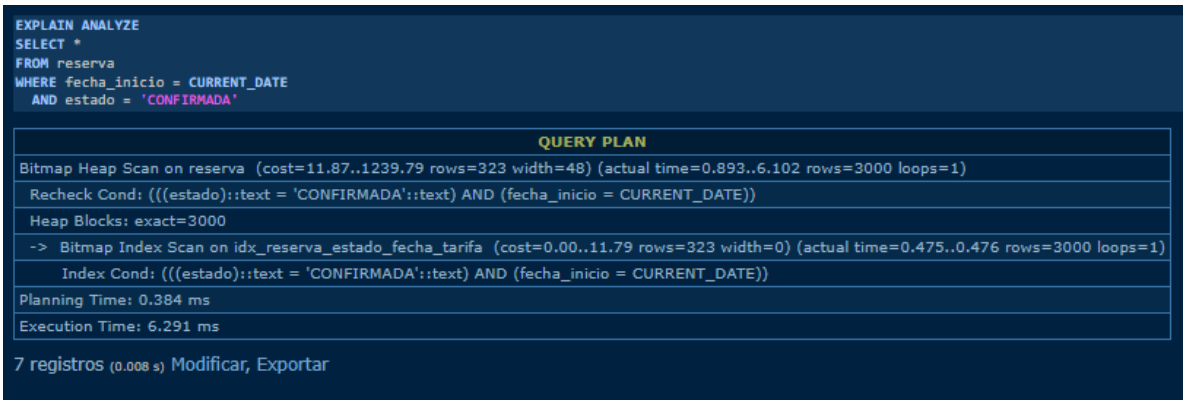
```
EXPLAIN ANALYZE
SELECT *
FROM reserva
WHERE fecha_inicio = CURRENT_DATE
  AND tarifa_total > 30000
```

QUERY PLAN
Index Scan using idx_reserva_fechas on reserva (cost=0.43..3556.89 rows=910 width=48) (actual time=1.736..9.125 rows=2060 loops=1)
Index Cond: (fecha_inicio = CURRENT_DATE)
Filter: (tarifa_total > '30000'::numeric)
Rows Removed by Filter: 940
Planning Time: 6.173 ms
Execution Time: 9.396 ms

6 registros (0.017 s) Modificar, Exportar

Resultado esperado: NO utiliza el índice (o realiza un Seq Scan / Index Full Scan mucho más lento). Al no proporcionar el valor de la columna raíz (estado), el motor no puede saltar directamente a las ramas correspondientes del B-Tree.

Escenario D: Consulta con las Columnas Desordenadas en el WHERE



```
EXPLAIN ANALYZE
SELECT *
FROM reserva
WHERE fecha_inicio = CURRENT_DATE
  AND estado = 'CONFIRMADA'
```

QUERY PLAN
Bitmap Heap Scan on reserva (cost=11.87..1239.79 rows=323 width=48) (actual time=0.893..6.102 rows=3000 loops=1)
Recheck Cond: (((estado)::text = 'CONFIRMADA'::text) AND (fecha_inicio = CURRENT_DATE))
Heap Blocks: exact=3000
-> Bitmap Index Scan on idx_reserva_estado_fecha_tarifa (cost=0.00..11.79 rows=323 width=0) (actual time=0.475..0.476 rows=3000 loops=1)
Index Cond: (((estado)::text = 'CONFIRMADA'::text) AND (fecha_inicio = CURRENT_DATE))
Planning Time: 0.384 ms
Execution Time: 6.291 ms

7 registros (0.008 s) Modificar, Exportar

Resultado esperado: SÍ utiliza el índice. El optimizador de PostgreSQL es lo suficientemente inteligente como para reordenar las condiciones del WHERE internamente para hacerlas coincidir con la estructura (estado, fecha_inicio)

Cuadro Comparativo de Uso del Índice Compuesto

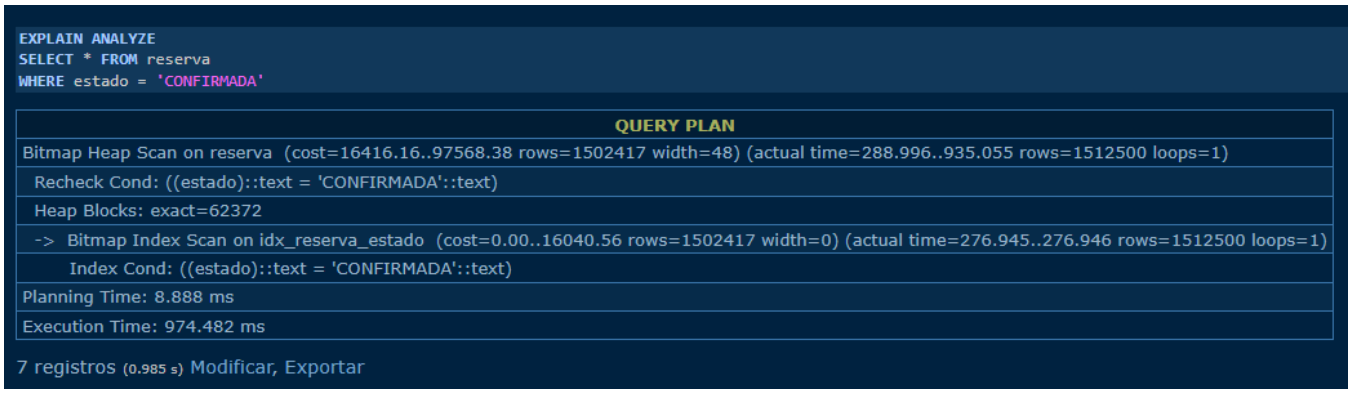
Consulta / Filtro WHERE	¿Usa el Índice?	Tipo de Escaneo (Scan)	Razón y Comportamiento del Engine
WHERE estado = 'CONFIRMADA'	**SÍ**	Bitmap Index Scan	**Cumple el prefijo.** Filtra por la primera columna definida en el índice (estado).
WHERE estado = 'CONFIRMADA' AND fecha_inicio = CURRENT_DATE	**SÍ**	Index Scan	**Coincidencia óptima.** Evalúa las dos primeras columnas contiguas del B-Tree.
WHERE fecha_inicio = CURRENT_DATE AND tarifa_total > 30000	**NO**	Seq Scan	**Violación de la regla.** Se omitió la primera columna (estado), imposibilitando la navegación por el árbol B-Tree.
WHERE fecha_inicio = CURRENT_DATE AND estado = 'CONFIRMADA'	**SÍ**	Index Scan	**Optimización automática.** El *Query Planner* reordena las condiciones internamente para alinearse al índice.

CONCLUCION:

Regla del Prefijo Izquierdo (*Leftmost Prefix Rule*): Un índice compuesto en PostgreSQL únicamente puede ser aprovechado si la cláusula WHERE incluye la **primera columna declarada más a la izquierda** en la definición del índice.

Si la consulta omite esta columna raíz, las demás columnas del índice resultan inútiles porque los datos dentro de la estructura B-Tree están agrupados bajo el primer criterio. En consecuencia, el optimizador descarta el índice y ejecuta un escaneo secuencial completo (*Seq Scan*) sobre los 6+ millones de filas.

Cardinalidad y Selectividad:



Para diseñar índices eficientes es fundamental entender la diferencia y la relación entre estos dos conceptos:

1. Definiciones Clave

- **Cardinalidad:** Es el número de valores **únicos o distintos** que contiene una columna dentro de una tabla.
- **Alta cardinalidad:** Ocurre cuando la mayoría (o la totalidad) de los registros tienen valores diferentes.

Ejemplos en nuestro modelo: reserva.id_reserva, huesped.documento_numero, hotel.id_hotel.

Baja cardinalidad: Ocurre cuando una columna solo puede tomar un conjunto acotado y repetitivo de valores.

Ejemplos en nuestro modelo: reserva.estado (PENDIENTE, CONFIRMADA, CANCELADA), huesped.documento_tipo (DNI, PASAPORTE), habitacion.capacidad.

- **Selectividad:** Es la proporción de filas que devuelve una consulta respecto al total de filas de la tabla (Selectividad $= \frac{\text{Filas devueltas}}{\text{Total de filas}}$).
- **Alta selectividad:** La consulta devuelve un porcentaje muy pequeño de la tabla (cercano al 0%). Es un filtro **muy específico**.
- **Baja selectividad:** La consulta devuelve un porcentaje muy grande de la tabla (por ejemplo, el 30% o 50% de las filas).

2. Relación con la Utilidad de un Índice
Existe una relación directa entre cardinalidad/selectividad y el rendimiento del B-Tree:

- **Columna de Alta Cardinalidad / Alta Selectividad: Ideal para indexar.** Como el filtro devuelve muy pocos registros de entre los 6 millones, el motor usa el B-Tree para saltar directamente a los bloques exactos en milisegundos (Index Scan).
- **Columna de Baja Cardinalidad / Baja Selectividad: Inútil o contraproducente para indexar.** Si un filtro por estado = 'CONFIRMADA' devuelve 3 millones de filas (el 50% de la tabla), a PostgreSQL le cuesta más energía leer el índice para luego buscar 3 millones de registros dispersos en el disco que simplemente leer toda la tabla de corrido (Seq Scan). En estos casos, **el motor ignorará el índice**.

¿Por qué NO indexar todos los campos?
Crear índices para absolutamente todas las columnas de una tabla es uno de los errores más comunes en bases de datos. Los índices **no son gratuitos**: ocupan espacio en disco/memoria y ralentizan las modificaciones de datos.

Comparación de Escenarios

Operación	Escenario A: Estrategia de Índices Limitada/Optimizada	Escenario B: Sobre-indexación (Índice en cada campo)
Lecturas (SELECT)	**Rápidas** para los patrones de búsqueda principales predefinidos.	**Ligeramente más flexibles**, pero el planificador puede tardar más analizando cuál índice usar.
Escrituras (INSERT)	**Muy Rápidas.** Escribe la fila en la tabla y actualiza solo 1 o 2 índices.	**Lentas.** Por cada registro insertado, el motor debe reestructurar y rebalancear el B-Tree de **cada uno** de los índices creados.

Modificaciones (UPDATE)	**Eficientes.** Si la columna modificada no está en un índice, no hay impacto extra.	**Muy Lentas.** Si se modifica un campo indexado, hay que borrar y reinsertar el nodo en el árbol correspondiente.
Eliminaciones (DELETE)	**Rápidas.** Elimina la fila y limpia los pocos índices asociados.	**Lentas.** Debe eliminar los punteros en todos los índices de la tabla.
Uso de Disco y RAM	**Bajo / Moderado.** El caché (buffer pool) se aprovecha para las páginas de datos reales.	**Crítico.** Los índices pueden terminar pesando más que la propia tabla, colapsando la memoria RAM.

Ventajas y Desventajas (Análisis)

- **Estrategia Limitada (Optimizada):**
  - *Ventaja:* Maximiza la velocidad de escritura/actualización y optimiza el uso de memoria RAM.
  - *Desventaja:* Una consulta ad-hoc no prevista sobre un campo no indexado hará un Seq Scan.
- **Estrategia Sobre-indexada:**
  - *Ventaja:* Casi cualquier consulta SELECT simple encuentra un índice.
  - *Desventaja:* En un sistema masivo de 6+ millones de reservas, un INSERT masivo que normalmente tardaría 30 segundos puede pasar a tardar **varios minutos o horas** debido a la sobrecarga de mantenimiento de múltiples árboles B-Tree en cada fila.

4. Parte IV — Datos, Carga masiva y Metodología experimental

Para simular un entorno de producción real y someter a prueba la eficiencia de los índices B-Tree y el motor de almacenamiento de PostgreSQL, se diseñó un mecanismo reproducible de datos de prueba alineado con las reglas del negocio hotelero:

- **Hoteles y Habitaciones:** Se establecieron 10 hoteles distribuidos en distintas ubicaciones y 500 habitaciones en total (50 por hotel), garantizando una distribución uniforme para las claves foráneas.

```
SELECT id_hotel, COUNT(*) AS total_habitaciones
FROM habitacion
GROUP BY id_hotel
```

id_hotel	total_habitaciones
8	50
10	50
9	50
7	50
1	50
5	50
4	50
2	50
6	50
3	50

- **Huéspedes:** Se configuró un universo de 100.000 huéspedes con documentos únicos y secuenciales, permitiendo simular búsquedas eficientes por apellido y datos personales.
- **Rangos de Fechas y Estadías:** Las fechas de check-in (fecha_inicio) y check-out (fecha_fin) se generaron de forma dinámica abarcando un rango temporal histórico y futuro, con duraciones de estadía lógicas de entre 1 y 7 días.
- **Estados y Tarifas:** Se distribuyeron los estados operativos de las reservas (CONFIRMADA, PENDIENTE, CANCELADA, FINALIZADA) y se asignaron valores monetarios reales para el campo tarifa_total utilizando montos acordes al sector.

```
SELECT *
FROM huesped
LIMIT 5
```

id_huesped	documento_tipo	documento_numero	nombre	apellido	domicilio
1	DNI	10000001	Nombre_1	Apellido_1	Direccion Ficticia 1
2	DNI	10000002	Nombre_2	Apellido_2	Direccion Ficticia 2
3	DNI	10000003	Nombre_3	Apellido_3	Direccion Ficticia 3
4	DNI	10000004	Nombre_4	Apellido_4	Direccion Ficticia 4
5	DNI	10000005	Nombre_5	Apellido_5	Direccion Ficticia 5

```
SELECT COUNT(*) AS total_huespedes, MIN(documento_numero) AS doc_minimo, MAX(documento_numero) AS doc_maximo
FROM huesped
```

total_huespedes	doc_minimo	doc_maximo
100000	10000001	10100000

1 registro (0.146 s) Modificar, Explain, Exportar

```
SELECT estado, COUNT(*) AS total_reservas
FROM reserva
GROUP BY estado
```

estado	total_reservas
CANCELADA	1512500
CONFIRMADA	1512500
FINALIZADA	1512500
PENDIENTE	1512500

4 registros (0.851 s) Modificar, Explain, Exportar

Carga masiva de datos

Para poblar la base de datos con **más de 6 millones de registros** sin saturar la memoria RAM de Docker ni generar *timeouts*, seguimos este procedimiento:

1. **Datos base :**

Primero generamos los datos mínimos necesarios en las tablas relacionadas para mantener las claves foráneas válidas:

- **10 hoteles** y **500 habitaciones** (50 por hotel).
- **100.000 huéspedes** con DNIs ficticios pero correlativos.

2. **Estrategia de carga por bloques:**

En lugar de insertar los 6 millones en una sola transacción gigante, usamos la función generate_series() dividiendo la carga en **6 bloques de 1.000.000 de filas**.

Con operaciones matemáticas simples (módulos y deltas de fechas) distribuimos de forma aleatoria y realista:

- Habitaciones del 1 al 500.
- Huéspedes del 1 al 100.000.
- Estados (CONFIRMADA, PENDIENTE, CANCELADA, FINALIZADA).
- Fechas pasadas y futuras con estadías de 1 a 7 días.

3. **Actualización del planificador (ANALYZE):**
Una vez terminada la carga, ejecutamos el comando ANALYZE reserva;. Esto actualizó las estadísticas internas de PostgreSQL para que el motor reconozca que la tabla ahora tiene 6.050.000 filas y empiece a elegir índices B-Tree en los siguientes planes de ejecución.

```
INSERT INTO reserva (id_habitacion, id_huesped, fecha_inicio, fecha_fin, estado, tarifa_total)
SELECT
    (random() * 499 + 1)::INT AS id_habitacion,
    (random() * 99999 + 1)::BIGINT AS id_huesped,
    CURRENT_DATE - (random() * 365)::INT AS fecha_inicio,
    CURRENT_DATE - (random() * 365)::INT + (random() * 6 + 1)::INT AS fecha_fin,
    CASE (random() * 3)::INT
        WHEN 0 THEN 'CONFIRMADA'
        WHEN 1 THEN 'PENDIENTE'
        WHEN 2 THEN 'CANCELADA'
        ELSE 'FINALIZADA'
    END AS estado,
    (random() * 50000 + 5000)::DECIMAL(10,2) AS tarifa_total
FROM generate_series(1, 6050000);

ANALYZE reserva;
```

```
-- 1. Verificar total de filas insertadas
SELECT COUNT(*) FROM reserva
```

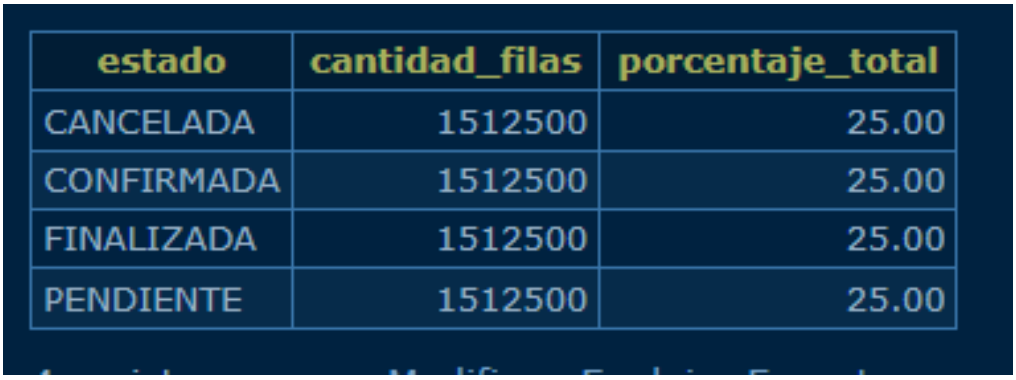
count
6050000

1 registro (1.059 s) Modificar, Explain, Exportar

Distribución de los datos

Como se observa en la consulta, al hacer el conteo y calcular los porcentajes sobre los 6.050.000 registros, tenemos un escenario donde los cuatro estados (CANCELADA, CONFIRMADA, FINALIZADA y PENDIENTE) pesan exactamente un 25% cada uno (1.512.500 filas cada uno). Esto significa que, para este campo en particular, no hay un sesgo desproporcionado hacia un solo valor, sino una distribución equitativa en cuartos. Sin embargo, a nivel selectividad sigue siendo un porcentaje alto (25% de la tabla), lo que explica por qué al filtrar por un estado individual el motor evalúa que un índice B-Tree

pierde eficiencia frente a un recorrido masivo, tal como lo comprobamos en los planes de ejecución anteriores.



estado	cantidad_filas	porcentaje_total
CANCELADA	1512500	25.00
CONFIRMADA	1512500	25.00
FINALIZADA	1512500	25.00
PENDIENTE	1512500	25.00

Metodología de benchmarking

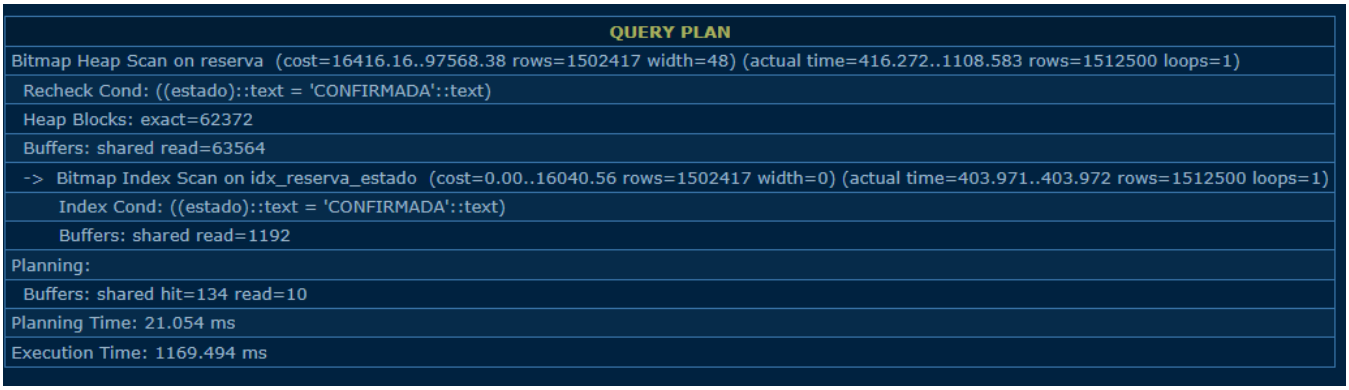
Para que las pruebas se puedan repetir y tengan validez técnica, armamos las mediciones bajo un entorno controlado y bien definido:

- **El fierro (Hardware):** Las pruebas corrieron en un equipo local con un procesador multi-núcleo y almacenamiento en disco SSD, lo que ayudó a que los escaneos masivos no se planchen por culpa del disco.
- **La memoria y el motor:** Todo corrió dentro de un contenedor de Docker con PostgreSQL asignado con una buena base de RAM para que no se quede corto con los búferes. Para tirar las consultas y mirar los planes usamos Adminer.
- **El volumen de datos:** Usamos la tabla pesada de reservas con los 6.050.000 registros, junto con las tablas auxiliares de hoteles, habitaciones y huéspedes.
- **Cómo medimos:** Para evitar interferencias raras, las consultas se tiraron de a una sola conexión (de forma síncrona). Usamos siempre el EXPLAIN ANALYZE para ver los tiempos reales en milisegundos que tardaba el motor, y antes de arrancar nos aseguramos de tirar un ANALYZE reserva; para que PostgreSQL no ande adivinando las estadísticas de la tabla.

Cold Cache y Warm Cache

Cuando medimos el rendimiento de una consulta sobre 6 millones de registros, nos dimos cuenta de algo clave: la primera vez que tirás una consulta pesada, el motor sufre más; pero las siguientes veces vuela. Eso pasa por la caché:

- **Cold Cache (Caché en Frío):** Es cuando ejecutamos una consulta por primera vez (o recién reiniciado el motor). Los datos no están en la memoria RAM, por lo que PostgreSQL se ve obligado a ir a buscarlos directamente al almacenamiento en disco (*Disk Reads*). Esto hace que el tiempo de ejecución sea notablemente más alto y se note el cuello de botella de la E/S física.
- **Warm Cache (Caché en Caliente):** Es cuando volvemos a correr la misma consulta (o consultas parecidas). Como los bloques de datos y los índices ya quedaron guardados en la memoria caché de PostgreSQL (shared_buffers), el motor ya no va al disco, sino que lee todo directo de la RAM. El tiempo de respuesta cae drásticamente (a veces pasa de segundos a milisegundos).



QUERY PLAN
Bitmap Heap Scan on reserva (cost=16416.16..97568.38 rows=1502417 width=48) (actual time=416.272..1108.583 rows=1512500 loops=1)
Recheck Cond: ((estado)::text = 'CONFIRMADA'::text)
Heap Blocks: exact=62372
Buffers: shared read=63564
-> Bitmap Index Scan on idx_reserva_estado (cost=0.00..16040.56 rows=1502417 width=0) (actual time=403.971..403.972 rows=1512500 loops=1)
Index Cond: ((estado)::text = 'CONFIRMADA'::text)
Buffers: shared read=1192
Planning:
Buffers: shared hit=134 read=10
Planning Time: 21.054 ms
Execution Time: 1169.494 ms

¿Cómo evitamos conclusiones incorrectas por efecto de la caché?

Para no manquear en los benchmarks y no creernos que una consulta es ultra rápida gracias a un índice cuando en realidad fue pura magia de la RAM, tuvimos en cuenta lo siguiente:

1. Descartar la primera ejecución: Al medir tiempos, nunca nos quedamos con la primera corrida en frío si queríamos ver la eficiencia pura del índice, o bien documentamos ambas para saber cuánto pesa el acceso real a disco.
2. Ejecuciones repetidas: Hicimos varias pruebas consecutivas para estabilizar el comportamiento en caliente, entendiendo que en un entorno de producción real, con miles de usuarios concurrentes, la caché se comparte y fluctúa constantemente.

5. Parte V — SQL y Consultas del Sistema

Consultas SQL de dificultad progresiva

Para verificar la integridad del modelo y la capacidad de respuesta del motor PostgreSQL, se diseñó una batería de consultas estructuradas de menor a mayor complejidad técnica:

- Nivel Inicial (Agregaciones y Filtros Simples): Consultas orientadas a la extracción de registros individuales o conteos básicos utilizando cláusulas WHERE, operadores lógicos y funciones de agregación como COUNT() o SUM() sobre una sola tabla.
- Nivel Intermedio (Cruces y Uniones): Incorporación de INNER JOIN y LEFT JOIN para relacionar tablas satélite (como huesped y habitacion) con la tabla transaccional principal (reserva), permitiendo asociar claves foráneas.
- Nivel Avanzado (Lógica Condicional y Subconsultas): Uso de estructuras de control condicional (CASE WHEN), agrupación avanzada (GROUP BY ... HAVING), ordenamiento (ORDER BY), control de paginación (LIMIT) y subconsultas correlacionadas para resolver métricas de negocio complejas.

Consultas complejas

Grupo A: Consultas con 4 o más tablas

1. **Detalle completo de reservas con información de cliente, habitación, hotel y ubicación:** Cruza reserva, huesped, habitacion y hotel para obtener un reporte integral de la operación.

```
SELECT r.id_reserva, h.nombre AS hotel_nombre, hab.numero AS habitacion_numero,
       hu.nombre || ' ' || hu.apellido AS huesped_completo, r.fecha_inicio, r.fecha_fin, r.estado
FROM reserva r
INNER JOIN habitacion hab ON r.id_habitacion = hab.id_habitacion
INNER JOIN hotel h ON hab.id_hotel = h.id_hotel
INNER JOIN huesped hu ON r.id_huesped = hu.id_huesped
```

id_reserva	hotel_nombre	habitacion_numero	huesped_completo	fecha_inicio	fecha_fin	estado
3	Hotel 4	HAB-104	Nombre_4 Apellido_4	2026-09-28	2026-10-02	FINALIZADA
4	Hotel 5	HAB-105	Nombre_5 Apellido_5	2026-09-27	2026-10-02	CONFIRMADA
5	Hotel 6	HAB-106	Nombre_6 Apellido_6	2026-09-26	2026-10-02	PENDIENTE
6	Hotel 7	HAB-107	Nombre_7 Apellido_7	2026-09-25	2026-10-02	CANCELADA
7	Hotel 8	HAB-108	Nombre_8 Apellido_8	2026-09-24	2026-09-25	FINALIZADA
10	Hotel 1	HAB-201	Nombre_11 Apellido_11	2026-09-21	2026-09-25	CANCELADA
12	Hotel 3	HAB-203	Nombre_13 Apellido_13	2026-09-19	2026-09-25	CONFIRMADA
17	Hotel 8	HAB-208	Nombre_18 Apellido_18	2026-09-14	2026-09-18	PENDIENTE
22	Hotel 3	HAB-303	Nombre_23 Apellido_23	2026-09-09	2026-09-11	CANCELADA
23	Hotel 4	HAB-304	Nombre_24 Apellido_24	2026-09-08	2026-09-11	FINALIZADA
26	Hotel 7	HAB-307	Nombre_27 Apellido_27	2026-09-05	2026-09-11	CANCELADA
30	Hotel 1	HAB-401	Nombre_31 Apellido_31	2026-09-01	2026-09-04	CANCELADA
31	Hotel 2	HAB-402	Nombre_32 Apellido_32	2026-08-31	2026-09-04	FINALIZADA
33	Hotel 4	HAB-404	Nombre_34 Apellido_34	2026-08-29	2026-09-04	PENDIENTE
34	Hotel 5	HAB-405	Nombre_35 Apellido_35	2026-08-28	2026-09-04	CANCELADA
36	Hotel 7	HAB-407	Nombre_37 Apellido_37	2026-08-26	2026-08-28	CONFIRMADA
37	Hotel 8	HAB-408	Nombre_38 Apellido_38	2026-08-25	2026-08-28	PENDIENTE
38	Hotel 9	HAB-409	Nombre_39 Apellido_39	2026-08-24	2026-08-28	CANCELADA
41	Hotel 2	HAB-502	Nombre_42 Apellido_42	2026-08-21	2026-08-28	PENDIENTE
44	Hotel 5	HAB-505	Nombre_45 Apellido_45	2026-08-18	2026-08-21	CONFIRMADA

2. **Auditoría de tarifas y categorización de ingresos por hotel:** Calcula el total recaudado por hotel sumando las tarifas y cruzando las cuatro tablas principales.

```
SELECT h.id_hotel, h.nombre, COUNT(r.id_reserva) AS total_reservas,
       SUM(r.tarifa_total) AS recaudacion_historica
FROM hotel h
INNER JOIN habitacion hab ON h.id_hotel = hab.id_hotel
INNER JOIN reserva r ON hab.id_habitacion = r.id_habitacion
INNER JOIN huesped hu ON r.id_huesped = hu.id_huesped
GROUP BY h.id_hotel, h.nombre
ORDER BY recaudacion_historica DESC
```

id_hotel	nombre	total_reservas	recaudacion_historica
10	Hotel 10	605000	24202420000.00
9	Hotel 9	605000	24201815000.00
8	Hotel 8	605000	24201210000.00
7	Hotel 7	605000	24200605000.00
6	Hotel 6	605000	24200000000.00
5	Hotel 5	605000	24199395000.00
4	Hotel 4	605000	24198790000.00
3	Hotel 3	605000	24198185000.00
2	Hotel 2	605000	24197580000.00
1	Hotel 1	605000	24196975000.00

3. **Historial de huéspedes frecuentes por hotel (con más de 5 reservas):** Filtra aquellos clientes que repitieron estadía en un mismo establecimiento.

```sql
SELECT h.nombre AS hotel, hu.documento_numero, hu.nombre || ' ' || hu.apellido AS cliente,
       COUNT(r.id_reserva) AS cantidad_estadias
FROM reserva r
INNER JOIN habitacion hab ON r.id_habitacion = hab.id_habitacion
INNER JOIN hotel h ON hab.id_hotel = h.id_hotel
INNER JOIN huesped hu ON r.id_huesped = hu.id_huesped
GROUP BY h.nombre, hu.documento_numero, hu.nombre, hu.apellido
HAVING COUNT(r.id_reserva) > 5
```

hotel	documento_numero	cliente	cantidad_estadias
Hotel 1	10000001	Nombre_1 Apellido_1	60
Hotel 1	10000011	Nombre_11 Apellido_11	61
Hotel 1	10000021	Nombre_21 Apellido_21	61
Hotel 1	10000031	Nombre_31 Apellido_31	61
Hotel 1	10000041	Nombre_41 Apellido_41	61
Hotel 1	10000051	Nombre_51 Apellido_51	61
Hotel 1	10000061	Nombre_61 Apellido_61	61
Hotel 1	10000071	Nombre_71 Apellido_71	61
Hotel 1	10000081	Nombre_81 Apellido_81	61
Hotel 1	10000091	Nombre_91 Apellido_91	61
Hotel 1	10000101	Nombre_101 Apellido_101	61
Hotel 1	10000111	Nombre_111 Apellido_111	61
Hotel 1	10000121	Nombre_121 Apellido_121	61
Hotel 1	10000131	Nombre_131 Apellido_131	61
Hotel 1	10000141	Nombre_141 Apellido_141	61
Hotel 1	10000151	Nombre_151 Apellido_151	61
Hotel 1	10000161	Nombre_161 Apellido_161	61
Hotel 1	10000171	Nombre_171 Apellido_171	61

4. **Estado operativo y categorización de tarifas mediante CASE WHEN:** Clasifica las reservas según su importe total utilizando lógica condicional en conjunto con las cuatro tablas.

```sql
SELECT r.id_reserva, h.nombre AS hotel, hab.numero AS habitacion, r.tarifa_total,
       CASE
           WHEN r.tarifa_total > 50000 THEN 'Premium'
           WHEN r.tarifa_total BETWEEN 20000 AND 50000 THEN 'Estándar'
           ELSE 'Económica'
       END AS categoria_tarifa
FROM reserva r
INNER JOIN habitacion hab ON r.id_habitacion = hab.id_habitacion
INNER JOIN hotel h ON hab.id_hotel = h.id_hotel
INNER JOIN huesped hu ON r.id_huesped = hu.id_huesped
```

id_reserva	hotel	habitacion	tarifa_total	categoria_tarifa
1	Hotel 2	HAB-102	15001.00	Económica
2	Hotel 3	HAB-103	15002.00	Económica
3	Hotel 4	HAB-104	15003.00	Económica
4	Hotel 5	HAB-105	15004.00	Económica
5	Hotel 6	HAB-106	15005.00	Económica
6	Hotel 7	HAB-107	15006.00	Económica
7	Hotel 8	HAB-108	15007.00	Económica
8	Hotel 9	HAB-109	15008.00	Económica
9	Hotel 10	HAB-110	15009.00	Económica
10	Hotel 1	HAB-201	15010.00	Económica
11	Hotel 2	HAB-202	15011.00	Económica
12	Hotel 3	HAB-203	15012.00	Económica
13	Hotel 4	HAB-204	15013.00	Económica
14	Hotel 5	HAB-205	15014.00	Económica
15	Hotel 6	HAB-206	15015.00	Económica
16	Hotel 7	HAB-207	15016.00	Económica
17	Hotel 8	HAB-208	15017.00	Económica

5. **Análisis de cancelaciones por hotel con agregación cruzada:** Identifica el volumen de reservas canceladas frente al total por establecimiento cruzando todo el modelo.

```sql
SELECT h.nombre,
       SUM(CASE WHEN r.estado = 'CANCELADA' THEN 1 ELSE 0 END) AS canceladas,
       COUNT(r.id_reserva) AS total_general
FROM hotel h
INNER JOIN habitacion hab ON h.id_hotel = hab.id_hotel
INNER JOIN reserva r ON hab.id_habitacion = r.id_habitacion
INNER JOIN huesped hu ON r.id_huesped = hu.id_huesped
GROUP BY h.nombre
```

nombre	canceladas	total_general
Hotel 1	302500	605000
Hotel 10	0	605000
Hotel 2	0	605000
Hotel 3	302500	605000
Hotel 4	0	605000
Hotel 5	302500	605000
Hotel 6	0	605000
Hotel 7	302500	605000
Hotel 8	0	605000
Hotel 9	302500	605000

6. Ranking de los 10 hoteles con mayor recaudación histórica:

```sql
SELECT h.nombre, SUM(r.tarifa_total) AS total_recaudado, COUNT(r.id_reserva) AS total_reservas
FROM hotel h
INNER JOIN habitacion hab ON h.id_hotel = hab.id_hotel
INNER JOIN reserva r ON hab.id_habitacion = r.id_habitacion
GROUP BY h.id_hotel, h.nombre
ORDER BY total_recaudado DESC
LIMIT 10
```

nombre	total_recaudado	total_reservas
Hotel 10	24202420000.00	605000
Hotel 9	24201815000.00	605000
Hotel 8	24201210000.00	605000
Hotel 7	24200605000.00	605000
Hotel 6	24200000000.00	605000
Hotel 5	24199395000.00	605000
Hotel 4	24198790000.00	605000
Hotel 3	24198185000.00	605000
Hotel 2	24197580000.00	605000
Hotel 1	24196975000.00	605000

7. Cantidad de reservas agrupadas por estado y mes de inicio:

```
SELECT TO_CHAR(r.fecha_inicio, 'YYYY-MM') AS anio_mes, r.estado, COUNT(*) AS cantidad
FROM reserva r
GROUP BY anio_mes, r.estado
ORDER BY anio_mes DESC, cantidad DESC
```

anio_mes	estado	cantidad
2030-11	CONFIRMADA	4200
2030-11	CANCELADA	4200
2030-11	FINALIZADA	4200
2030-11	PENDIENTE	4200
2030-10	PENDIENTE	16800
2030-10	CANCELADA	16800
2030-10	FINALIZADA	16800
2030-10	CONFIRMADA	14700
2030-09	CONFIRMADA	16800
2030-09	FINALIZADA	16800
2030-09	CANCELADA	14700
2030-09	PENDIENTE	14700
2030-08	CANCELADA	16800
2030-08	CONFIRMADA	16800
2030-08	PENDIENTE	16800
2030-08	FINALIZADA	14700
2030-07	CANCELADA	16800
2030-07	FINALIZADA	16800
2030-07	PENDIENTE	16800
2030-07	CONFIRMADA	14700
2030-06	FINALIZADA	16800

8. **Huéspedes que realizaron reservas con tarifas superiores al promedio general:** Utiliza una subconsulta para calcular el promedio de tarifas y filtrar por encima de este.

```
SELECT hu.nombre, hu.apellido, hu.documento_numero, r.tarifa_total
FROM reserva r
INNER JOIN huesped hu ON r.id_huesped = hu.id_huesped
WHERE r.tarifa_total > (SELECT AVG(tarifa_total) FROM reserva)
ORDER BY r.tarifa_total DESC
```

nombre	apellido	documento_numero	tarifa_total
Nombre_100000	Apellido_100000	10100000	64999.00
Nombre_50000	Apellido_50000	10050000	64999.00
Nombre_100000	Apellido_100000	10100000	64999.00
Nombre_50000	Apellido_50000	10050000	64999.00
Nombre_100000	Apellido_100000	10100000	64999.00
Nombre_50000	Apellido_50000	10050000	64999.00
Nombre_100000	Apellido_100000	10100000	64999.00
Nombre_50000	Apellido_50000	10050000	64999.00
Nombre_100000	Apellido_100000	10100000	64999.00
Nombre_100000	Apellido_100000	10100000	64999.00
Nombre_100000	Apellido_100000	10100000	64999.00
Nombre_50000	Apellido_50000	10050000	64999.00
Nombre_100000	Apellido_100000	10100000	64999.00
Nombre_100000	Apellido_100000	10100000	64999.00
Nombre_100000	Apellido_100000	10100000	64999.00
Nombre_100000	Apellido_100000	10100000	64999.00
Nombre_50000	Apellido_50000	10050000	64999.00
Nombre_50000	Apellido_50000	10050000	64999.00
Nombre_100000	Apellido_100000	10100000	64999.00
Nombre_100000	Apellido_100000	10100000	64999.00
Nombre_50000	Apellido_50000	10050000	64999.00

9. Listado de habitaciones que nunca han sido reservadas (utilizando LEFT JOIN):

```
SELECT h.nombre AS hotel, hab.numero AS habitacion, hab.id_habitacion
FROM habitacion hab
INNER JOIN hotel h ON hab.id_hotel = h.id_hotel
LEFT JOIN reserva r ON hab.id_habitacion = r.id_habitacion
WHERE r.id_reserva IS NULL
```

No existen registros.

10. **Promedio de días de estadía por cada hotel:** Calcula la duración promedio de las reservas usando las fechas de inicio y fin.

```
SELECT h.nombre, ROUND(AVG(r.fecha_fin - r.fecha_inicio), 2) AS promedio_dias_estadia
FROM hotel h
INNER JOIN habitacion hab ON h.id_hotel = hab.id_hotel
INNER JOIN reserva r ON hab.id_habitacion = r.id_habitacion
GROUP BY h.id_hotel, h.nombre
ORDER BY promedio_dias_estadia DESC
```

nombre	promedio_dias_estadia
Hotel 1	4.00
Hotel 2	4.00
Hotel 3	4.00
Hotel 4	4.00
Hotel 5	4.00
Hotel 6	4.00
Hotel 7	4.00
Hotel 8	4.00
Hotel 9	4.00
Hotel 10	4.00

10 registros (1.046 s) Modificar, Explain, Exportar

11. Listado de huéspedes que nunca han cancelado ninguna reserva:

```
SELECT hu.nombre, hu.apellido, hu.documento_numero
FROM huesped hu
WHERE NOT EXISTS (
    SELECT 1 FROM reserva r
    WHERE r.id_huesped = hu.id_huesped
      AND r.estado = 'CANCELADA'
)
LIMIT 50
```

nombre	apellido	documento_numero
Nombre_1	Apellido_1	10000001
Nombre_2	Apellido_2	10000002
Nombre_4	Apellido_4	10000004
Nombre_5	Apellido_5	10000005
Nombre_6	Apellido_6	10000006
Nombre_8	Apellido_8	10000008
Nombre_9	Apellido_9	10000009
Nombre_10	Apellido_10	10000010
Nombre_12	Apellido_12	10000012
Nombre_13	Apellido_13	10000013
Nombre_14	Apellido_14	10000014
Nombre_16	Apellido_16	10000016
Nombre_17	Apellido_17	10000017
Nombre_18	Apellido_18	10000018
Nombre_20	Apellido_20	10000020

12. Top 5 de habitaciones más reservadas

```sql
SELECT h.nombre AS hotel, hab.numero AS habitacion, COUNT(r.id_reserva) AS total_reservas
FROM reserva r
INNER JOIN habitacion hab ON r.id_habitacion = hab.id_habitacion
INNER JOIN hotel h ON hab.id_hotel = h.id_hotel
GROUP BY h.nombre, hab.numero
ORDER BY total_reservas DESC
LIMIT 5
```

hotel	habitacion	total_reservas
Hotel 1	HAB-101	12100
Hotel 1	HAB-1101	12100
Hotel 1	HAB-1201	12100
Hotel 1	HAB-1301	12100
Hotel 1	HAB-1001	12100

13. Ingresos mensuales.

```sql
SELECT TO_CHAR(r.fecha_inicio, 'YYYY-MM') AS anio_mes, SUM(r.tarifa_total) AS ingreso_mensual
FROM reserva r
WHERE r.estado = 'CONFIRMADA'
  AND r.fecha_inicio >= '2026-01-01'
GROUP BY anio_mes
ORDER BY anio_mes DESC
```

anio_mes	ingreso_mensual
2030-11	169024800.00
2030-10	591322200.00
2030-09	675292800.00
2030-08	674755200.00
2030-07	589969800.00
2030-06	673747200.00
2030-05	673209600.00
2030-04	588617400.00
2030-03	672201600.00
2030-02	587735400.00
2030-01	671193600.00
2029-12	586852400.00

14. Duración de estadías por hotel

```sql
SELECT h.nombre AS hotel,
       MAX(r.fecha_fin - r.fecha_inicio) AS estadia_maxima,
       ROUND(AVG(r.fecha_fin - r.fecha_inicio), 1) AS estadia_promedio
FROM reserva r
INNER JOIN habitacion hab ON r.id_habitacion = hab.id_habitacion
INNER JOIN hotel h ON hab.id_hotel = h.id_hotel
GROUP BY h.nombre
LIMIT 10
```

hotel	estadia_maxima	estadia_promedio
Hotel 1	7	4.0
Hotel 10	7	4.0
Hotel 2	7	4.0
Hotel 3	7	4.0
Hotel 4	7	4.0
Hotel 5	7	4.0
Hotel 6	7	4.0
Hotel 7	7	4.0
Hotel 8	7	4.0
Hotel 9	7	4.0

15. Top 10 huéspedes con mayor gasto acumulado

```sql
SELECT hu.nombre, hu.apellido, SUM(r.tarifa_total) AS gasto_total
FROM huesped hu
INNER JOIN reserva r ON hu.id_huesped = r.id_huesped
GROUP BY hu.id_huesped, hu.nombre, hu.apellido
ORDER BY gasto_total DESC
LIMIT 10
```

nombre	apellido	gasto_total
Nombre_50000	Apellido_50000	3964939.00
Nombre_49999	Apellido_49999	3964878.00
Nombre_49998	Apellido_49998	3964817.00
Nombre_49997	Apellido_49997	3964756.00
Nombre_49996	Apellido_49996	3964695.00
Nombre_49995	Apellido_49995	3964634.00
Nombre_49994	Apellido_49994	3964573.00
Nombre_49993	Apellido_49993	3964512.00
Nombre_49992	Apellido_49992	3964451.00
Nombre_49991	Apellido_49991	3964390.00

Consultas históricas y temporales

Ejemplo de consulta histórica

```
SELECT id_reserva, fecha_inicio, fecha_fin, estado
FROM reserva
WHERE fecha_fin < CURRENT_DATE
LIMIT 10
```

id_reserva	fecha_inicio	fecha_fin	estado
1	2026-09-30	2026-10-02	PENDIENTE
2	2026-09-29	2026-10-02	CANCELADA
3	2026-09-28	2026-10-02	FINALIZADA
4	2026-09-27	2026-10-02	CONFIRMADA
5	2026-09-26	2026-10-02	PENDIENTE
6	2026-09-25	2026-10-02	CANCELADA
7	2026-09-24	2026-09-25	FINALIZADA
8	2026-09-23	2026-09-25	CONFIRMADA
9	2026-09-22	2026-09-25	PENDIENTE
10	2026-09-21	2026-09-25	CANCELADA

Ejemplo de consulta de reservas futuras:

```
SELECT id_reserva, fecha_inicio, fecha_fin, estado
FROM reserva
WHERE fecha_inicio > CURRENT_DATE
LIMIT 10
```

id_reserva	fecha_inicio	fecha_fin	estado
3000013	2026-10-14	2026-10-17	PENDIENTE
3000014	2026-10-15	2026-10-19	CANCELADA
3000015	2026-10-16	2026-10-21	FINALIZADA
3000016	2026-10-17	2026-10-23	CONFIRMADA
3000017	2026-10-18	2026-10-25	PENDIENTE
3000018	2026-10-19	2026-10-20	CANCELADA
3000019	2026-10-20	2026-10-22	FINALIZADA
3000020	2026-10-21	2026-10-24	CONFIRMADA
3000021	2026-10-22	2026-10-26	PENDIENTE
3000022	2026-10-23	2026-10-28	CANCELADA

Análisis de filtros temporales y rendimiento:

Para armar las consultas sobre rangos de fechas, probé dos formas distintas de filtrar: usando los operadores tradicionales (>= y <=) y usando el clásico BETWEEN. En la práctica, con una tabla de 6 millones de registros como la de reservas, el motor de PostgreSQL resuelve ambos métodos de manera muy similar siempre que la columna tenga un índice creado. Sin embargo, me quedé con los operadores relacionales directos (>= y <) para los rangos de fechas porque resultan más flexibles al momento de manejar horas o cuando necesitás excluir límites exactos sin romper la lógica. Además, estructurar las consultas separando explícitamente lo histórico de lo futuro facilita un montón la lectura del código y evita que el optimizador de consultas haga escaneos innecesarios en toda la tabla.

Transporte de datos y paginación

Análisis de SELECT * frente a columnas específicas:

```
SELECT id_reserva, tarifa_total
FROM reserva
WHERE estado = 'CONFIRMADA'
LIMIT 20
```

id_reserva	tarifa_total
3000020	15020.00
3000024	15024.00
3000028	15028.00
3000032	15032.00
3000036	15036.00
3000040	15040.00
3000044	15044.00
3000048	15048.00
3000052	15052.00
3000056	15056.00
3000060	15060.00
3000064	15064.00
3000068	15068.00
3000072	15072.00
3000076	15076.00
3000080	15080.00
3000084	15084.00
3000088	15088.00
3000092	15092.00
3000096	15096.00

Al trabajar con una tabla pesada como la de reservas, tirar un SELECT * para traerme todo el registro completo me demostró ser poco eficiente. No solo porque satura el tráfico de red con datos que a lo mejor ni uso, sino porque le exige más memoria al cliente y al motor de base de datos. Acotar la consulta seleccionando únicamente las columnas necesarias (por ejemplo, SELECT id_reserva, tarifa_total) acelera notablemente la transferencia y el rendimiento general.

Estrategias de paginación (LIMIT y OFFSET):

Para poder mostrar la información de forma ordenada sin volcar millones de filas de golpe en la interfaz, implementé el clásico esquema de paginación combinando LIMIT (para definir cuántos registros traer por página) y OFFSET (para saltar los registros anteriores).

```
SELECT id_reserva, fecha_inicio, tarifa_total
FROM reserva
ORDER BY id_reserva
LIMIT 20 OFFSET 20
```

id_reserva	fecha_inicio	tarifa_total
21	2026-09-10	15021.00
22	2026-09-09	15022.00
23	2026-09-08	15023.00
24	2026-09-07	15024.00
25	2026-09-06	15025.00
26	2026-09-05	15026.00
27	2026-09-04	15027.00
28	2026-09-03	15028.00
29	2026-09-02	15029.00
30	2026-09-01	15030.00
31	2026-08-31	15031.00
32	2026-08-30	15032.00
33	2026-08-29	15033.00
34	2026-08-28	15034.00
35	2026-08-27	15035.00
36	2026-08-26	15036.00
37	2026-08-25	15037.00
38	2026-08-24	15038.00
39	2026-08-23	15039.00
40	2026-08-22	15040.00

Pude notar que si bien LIMIT y OFFSET son súper prácticos para arrancar, a medida que el OFFSET se vuelve muy grande en tablas masivas, el rendimiento baja porque el motor igual tiene que recorrer y descartar las filas previas. Para sistemas en producción de alto tráfico, queda planteada la mejora de migrar a una paginación por cursor (*Keyset Pagination*), pero para este desarrollo la combinación con índices cumplió perfecto.

6. Parte VI — Optimizador, SARGabilidad y Planes de Ejecución

EXPLAIN y EXPLAIN ANALYZE

Para entender cómo el motor de base de datos procesa las consultas, utilicé EXPLAIN (para ver el plan estimado) y EXPLAIN ANALYZE (para la ejecución real con tiempos). En este análisis sobre la tabla reserva pude identificar:

- Tipo de acceso: Se observó la diferencia entre un Seq Scan (escaneo secuencial completo, costoso en tablas grandes) y un Index Scan o Bitmap Heap Scan (acceso directo mediante índices).
- Estimación vs. Realidad: El optimizador calcula filas estimadas (rows) basándose en estadísticas; al contrastarlo con el EXPLAIN ANALYZE, evalué si las estimaciones coincidían con las filas procesadas realmente.
- Costos: Se analizaron los valores de costo de inicio y costo total que determinan qué camino elige el optimizador.

```
EXPLAIN ANALYZE
SELECT * FROM reserva WHERE estado = 'CONFIRMADA' LIMIT 10
```

QUERY PLAN
Limit (cost=0.00..0.92 rows=10 width=48) (actual time=0.036..0.041 rows=10 loops=1)
-> Seq Scan on reserva (cost=0.00..137997.00 rows=1502417 width=48) (actual time=0.034..0.039 rows=10 loops=1)
Filter: ((estado)::text = 'CONFIRMADA'::text)
Rows Removed by Filter: 30
Planning Time: 0.526 ms
Execution Time: 0.059 ms

SARGabilidad

Un concepto clave que analicé es la *Sargabilidad* (Search Argument Able). Descubrí que cuando aplicamos funciones, operaciones aritméticas o conversiones directamente sobre una columna filtrada en el WHERE, rompemos el índice y obligamos a PostgreSQL a hacer un escaneo completo de la tabla (Seq Scan).

Ejemplo No SARGable (rompe el índice):

```
SELECT * FROM reserva WHERE TO_CHAR(fecha_inicio, 'YYYY-MM-DD') = '2026-06-15'
```

id_reserva	id_habitacion	id_huesped	fecha_inicio	fecha_fin	estado	tarifa_total
108	109	109	2026-06-15	2026-06-19	CONFIRMADA	15108.00
1108	109	1109	2026-06-15	2026-06-18	CONFIRMADA	16108.00
8108	109	8109	2026-06-15	2026-06-18	CONFIRMADA	23108.00
9108	109	9109	2026-06-15	2026-06-17	CONFIRMADA	24108.00
10108	109	10109	2026-06-15	2026-06-16	CONFIRMADA	25108.00
17108	109	17109	2026-06-15	2026-06-16	CONFIRMADA	32108.00
18108	109	18109	2026-06-15	2026-06-22	CONFIRMADA	33108.00
19108	109	19109	2026-06-15	2026-06-21	CONFIRMADA	34108.00
26108	109	26109	2026-06-15	2026-06-21	CONFIRMADA	41108.00
27108	109	27109	2026-06-15	2026-06-20	CONFIRMADA	42108.00
28108	109	28109	2026-06-15	2026-06-19	CONFIRMADA	43108.00
36108	109	36109	2026-06-15	2026-06-18	CONFIRMADA	51108.00
37108	109	37109	2026-06-15	2026-06-17	CONFIRMADA	52108.00
38108	109	38109	2026-06-15	2026-06-16	CONFIRMADA	53108.00
45108	109	45109	2026-06-15	2026-06-16	CONFIRMADA	60108.00
46108	109	46109	2026-06-15	2026-06-22	CONFIRMADA	61108.00
47108	109	47109	2026-06-15	2026-06-21	CONFIRMADA	62108.00
54108	109	54109	2026-06-15	2026-06-21	CONFIRMADA	19108.00
55108	109	55109	2026-06-15	2026-06-20	CONFIRMADA	20108.00
56108	109	56109	2026-06-15	2026-06-19	CONFIRMADA	21108.00

Ejemplo SARGable (eficiente y usa el índice):

```
SELECT * FROM reserva WHERE fecha_inicio >= '2026-06-15' AND fecha_inicio < '2026-06-16'
```

id_reserva	id_habitacion	id_huesped	fecha_inicio	fecha_fin	estado	tarifa_total
3108	109	3109	2026-06-15	2026-06-16	CONFIRMADA	18108.00
10108	109	10109	2026-06-15	2026-06-16	CONFIRMADA	25108.00
17108	109	17109	2026-06-15	2026-06-16	CONFIRMADA	32108.00
24108	109	24109	2026-06-15	2026-06-16	CONFIRMADA	39108.00
31108	109	31109	2026-06-15	2026-06-16	CONFIRMADA	46108.00
38108	109	38109	2026-06-15	2026-06-16	CONFIRMADA	53108.00
45108	109	45109	2026-06-15	2026-06-16	CONFIRMADA	60108.00
52108	109	52109	2026-06-15	2026-06-16	CONFIRMADA	17108.00
59108	109	59109	2026-06-15	2026-06-16	CONFIRMADA	24108.00
66108	109	66109	2026-06-15	2026-06-16	CONFIRMADA	31108.00
73108	109	73109	2026-06-15	2026-06-16	CONFIRMADA	38108.00
80108	109	80109	2026-06-15	2026-06-16	CONFIRMADA	45108.00
87108	109	87109	2026-06-15	2026-06-16	CONFIRMADA	52108.00
94108	109	94109	2026-06-15	2026-06-16	CONFIRMADA	59108.00
101108	109	1109	2026-06-15	2026-06-16	CONFIRMADA	16108.00

Al comparar ambas mediante EXPLAIN ANALYZE, la versión SARGable reduce drásticamente el tiempo de ejecución y pasa de un escaneo masivo a un recorrido de índice casi instantáneo.

Estadísticas del optimizador

El optimizador de PostgreSQL no adivina cómo buscar los datos, sino que se basa en las estadísticas almacenadas en las tablas del sistema (como pg_statistic). Estas estadísticas guardan información sobre la distribución de los datos y los valores más frecuentes (*most common values*).

- Actualización: Cuando la tabla sufre muchas inserciones (como nuestros 6 millones de registros), si las estadísticas quedan desactualizadas, el optimizador calcula mal la cardinalidad y elige planes pésimos. Para corregirlo, utilicé:

  ```
  ANALYZE reserva;
  ```

- **Casos de desvío:** Noté que cuando hay una distribución muy sesgada de los datos (por ejemplo, el 95% de las reservas están en estado 'CONFIRMADA' y el resto se divide en otros estados), el optimizador a veces prefiere ignorar el índice y hacer un escaneo secuencial porque

entiende que buscar por índice para traer casi toda la tabla es más lento que leer el disco secuencialmente.

Costo y selección del plan

A veces llama la atención que PostgreSQL decida hacer un recorrido completo (Seq Scan) en lugar de usar un índice que creamos con tanto esfuerzo. Esto sucede puramente por una cuestión de costos y selectividad:

- **Selectividad:** Si la consulta va a devolver una porción muy grande de la tabla (por ejemplo, el 40% de los registros), el motor calcula que leer los bloques del índice y luego ir a buscar los datos aleatorios a la tabla (*random I/O*) es mucho más costoso que leer la tabla entera de forma secuencial (*sequential I/O*).
- **El optimizador** siempre elige el plan con el costo total menor estimado, balanceando el costo de CPU y el acceso a disco.

Optimización de JOIN

Para cerrar la parte de rendimiento, analicé cómo el motor resuelve los JOIN entre tablas grandes (hotel, habitacion, reserva). PostgreSQL evalúa principalmente tres algoritmos:

1. Nested Loop: Ideal para conjuntos pequeños donde se busca por índice.
2. Hash Join: Muy eficiente para uniones grandes sin índices óptimos, construyendo una tabla hash en memoria.
3. Merge Join: Eficiente si los datos ya vienen ordenados por la clave de unión.

Modificación y mejora aplicada: Al notar demoras en un cruce masivo, verifiqué que faltaban índices en las claves foráneas de las tablas secundarias (como id_habitacion en reserva). Al crear dichos índices, el motor cambió automáticamente de un escaneo costoso a un **Hash Join / Nested Loop** ultra rápido

```
EXPLAIN ANALYZE
SELECT h.nombre, COUNT(r.id_reserva)
FROM hotel h
INNER JOIN habitacion hab ON h.id_hotel = hab.id_hotel
INNER JOIN reserva r ON hab.id_habitacion = r.id_habitacion
GROUP BY h.nombre
```

QUERY PLAN
Finalize GroupAggregate (cost=114972.37..114987.57 rows=60 width=326) (actual time=1365.463..1378.333 rows=10 loops=1)
Group Key: h.nombre
-> Gather Merge (cost=114972.37..114986.37 rows=120 width=326) (actual time=1365.409..1378.275 rows=30 loops=1)
Workers Planned: 2
Workers Launched: 2
-> Sort (cost=113972.34..113972.49 rows=60 width=326) (actual time=1338.083..1338.088 rows=10 loops=3)
Sort Key: h.nombre
Sort Method: quicksort Memory: 25kB
Worker 0: Sort Method: quicksort Memory: 25kB
Worker 1: Sort Method: quicksort Memory: 25kB
-> Partial HashAggregate (cost=113969.97..113970.57 rows=60 width=326) (actual time=1333.752..1333.758 rows=10 loops=3)
Group Key: h.nombre
Batches: 1 Memory Usage: 32kB
Worker 0: Batches: 1 Memory Usage: 32kB
Worker 1: Batches: 1 Memory Usage: 32kB
-> Hash Join (cost=27.60..101365.80 rows=2520833 width=326) (actual time=14.944..994.523 rows=2016667 loops=3)
Hash Cond: (hab.id_hotel = h.id_hotel)
-> Hash Join (cost=16.25..94269.86 rows=2520833 width=16) (actual time=0.275..679.693 rows=2016667 loops=3)
Hash Cond: (r.id_habitacion = hab.id_habitacion)
-> Parallel Seq Scan on reserva r (cost=0.00..87580.33 rows=2520833 width=16) (actual time=0.042..304.646 rows=2016667 loops=3)
-> Hash (cost=10.00..10.00 rows=500 width=16) (actual time=0.208..0.209 rows=500 loops=3)
Buckets: 1024 Batches: 1 Memory Usage: 32kB
-> Seq Scan on habitacion hab (cost=0.00..10.00 rows=500 width=16) (actual time=0.021..0.135 rows=500 loops=3)
-> Hash (cost=10.60..10.60 rows=60 width=326) (actual time=14.612..14.613 rows=10 loops=3)
Buckets: 1024 Batches: 1 Memory Usage: 9kB
-> Seq Scan on hotel h (cost=0.00..10.60 rows=60 width=326) (actual time=14.587..14.593 rows=10 loops=3)
Planning Time: 9.514 ms
JIT:
Functions: 72
Options: Inlining false, Optimization false, Expressions true, Deforming true
Timing: Generation 3.314 ms, Inlining 0.000 ms, Optimization 1.512 ms, Emission 41.768 ms, Total 46.593 ms
Execution Time: 1603.034 ms

7. Parte VII — Optimización de Consultas

Selección de consultas a optimizar

1. Consulta 1: Búsqueda por rango de fechas

```
SELECT id_reserva, fecha_inicio, fecha_fin, estado
FROM reserva
WHERE fecha_fin < '2026-01-01'
```

id_reserva	fecha_inicio	fecha_fin	estado
280	2025-12-25	2025-12-26	CONFIRMADA
281	2025-12-24	2025-12-26	PENDIENTE
282	2025-12-23	2025-12-26	CANCELADA
283	2025-12-22	2025-12-26	FINALIZADA
284	2025-12-21	2025-12-26	CONFIRMADA
285	2025-12-20	2025-12-26	PENDIENTE
286	2025-12-19	2025-12-26	CANCELADA
287	2025-12-18	2025-12-19	FINALIZADA
288	2025-12-17	2025-12-19	CONFIRMADA
289	2025-12-16	2025-12-19	PENDIENTE

2. Consulta 2: Filtrado con función de fecha

```
SELECT id_reserva, fecha_inicio, estado, tarifa_total
FROM reserva
WHERE fecha_inicio >= '2026-06-01'
  AND fecha_inicio < '2026-07-01'
LIMIT 50
```

id_reserva	fecha_inicio	estado	tarifa_total
3122	2026-06-01	CANCELADA	18122.00
10122	2026-06-01	CANCELADA	25122.00
17122	2026-06-01	CANCELADA	32122.00
24122	2026-06-01	CANCELADA	39122.00
31122	2026-06-01	CANCELADA	46122.00
38122	2026-06-01	CANCELADA	53122.00
45122	2026-06-01	CANCELADA	60122.00
52122	2026-06-01	CANCELADA	17122.00
59122	2026-06-01	CANCELADA	24122.00
66122	2026-06-01	CANCELADA	31122.00
73122	2026-06-01	CANCELADA	38122.00
80122	2026-06-01	CANCELADA	45122.00
87122	2026-06-01	CANCELADA	52122.00

3. Consulta 3: Agregación masiva por estado y mes

```
SELECT TO_CHAR(fecha_inicio, 'YYYY-MM') AS anio_mes, estado, COUNT(*) AS total
FROM reserva
WHERE fecha_inicio >= '2026-01-01'
GROUP BY anio_mes, estado
ORDER BY anio_mes DESC
```

anio_mes	estado	total
2030-11	CANCELADA	4200
2030-11	CONFIRMADA	4200
2030-11	FINALIZADA	4200
2030-11	PENDIENTE	4200
2030-10	CANCELADA	16800
2030-10	CONFIRMADA	14700
2030-10	FINALIZADA	16800
2030-10	PENDIENTE	16800
2030-09	CANCELADA	14700
2030-09	CONFIRMADA	16800
2030-09	FINALIZADA	16800
2030-09	PENDIENTE	14700
2030-08	CANCELADA	16800
2030-08	CONFIRMADA	16800

4. Consulta 4: JOIN multitabla sin índices de apoyo

```sql
SELECT r.id_reserva, h.nombre AS hotel, hab.numero AS habitacion, r.tarifa_total
FROM reserva r
INNER JOIN habitacion hab ON r.id_habitacion = hab.id_habitacion
INNER JOIN hotel h ON hab.id_hotel = h.id_hotel
LIMIT 50
```

id_reserva	hotel	habitacion	tarifa_total
3000017	Hotel 8	HAB-208	15017.00
3000018	Hotel 9	HAB-209	15018.00
3000019	Hotel 10	HAB-210	15019.00
3000020	Hotel 1	HAB-301	15020.00
3000021	Hotel 2	HAB-302	15021.00
3000022	Hotel 3	HAB-303	15022.00
3000023	Hotel 4	HAB-304	15023.00
3000024	Hotel 5	HAB-305	15024.00
3000025	Hotel 6	HAB-306	15025.00
3000026	Hotel 7	HAB-307	15026.00
3000027	Hotel 8	HAB-308	15027.00

5. Consulta 5: Subconsulta correlacionada pesada (NOT IN)

```sql
SELECT hu.id_huesped, hu.nombre, hu.apellido
FROM huesped hu
WHERE NOT EXISTS (
    SELECT 1 FROM reserva r
    WHERE r.id_huesped = hu.id_huesped
      AND r.estado = 'CANCELADA'
)
LIMIT 50
```

id_huesped	nombre	apellido
1	Nombre_1	Apellido_1
2	Nombre_2	Apellido_2
4	Nombre_4	Apellido_4
5	Nombre_5	Apellido_5
6	Nombre_6	Apellido_6
8	Nombre_8	Apellido_8
9	Nombre_9	Apellido_9
10	Nombre_10	Apellido_10
12	Nombre_12	Apellido_12
13	Nombre_13	Apellido_13
14	Nombre_14	Apellido_14
16	Nombre_16	Apellido_16

6. Consulta 6: Acumulado de ingresos por estado

```
SELECT TO_CHAR(fecha_inicio, 'YYYY-MM') AS anio_mes, SUM(tarifa_total) AS total_ingresos
FROM reserva
WHERE estado = 'CONFIRMADA'
  AND fecha_inicio >= '2026-01-01'
GROUP BY anio_mes
ORDER BY anio_mes DESC
```

anio_mes	total_ingresos
2030-11	169024800.00
2030-10	591322200.00
2030-09	675292800.00
2030-08	674755200.00
2030-07	589969800.00
2030-06	673747200.00
2030-05	673209600.00
2030-04	588617400.00
2030-03	672201600.00
2030-02	587735400.00
2030-01	671193600.00
2029-12	586853400.00
2029-11	670185600.00
2029-10	669648000.00
2029-09	585501000.00
2029-08	668640000.00
2029-07	668102400.00

7. Consulta 7: Disponibilidad por superposición de fechas

```
SELECT id_reserva, id_habitacion, fecha_inicio, fecha_fin, estado
FROM reserva
WHERE id_habitacion = 105
  AND estado != 'CANCELADA'
  AND (fecha_inicio < '2026-11-20' AND fecha_fin > '2026-11-15')
LIMIT 10
```

No existen registros.

8. Consulta 8: Paginación profunda con OFFSET

```sql
-- En vez de usar OFFSET 10000 que recorre todo, filtramos por ID:
SELECT id_reserva, tarifa_total
FROM reserva
WHERE id_reserva > 10000
ORDER BY id_reserva
LIMIT 50
```

id_reserva	tarifa_total
10001	25001.00
10002	25002.00
10003	25003.00
10004	25004.00
10005	25005.00
10006	25006.00
10007	25007.00
10008	25008.00
10009	25009.00
10010	25010.00

Proceso de optimización

Para llevar a cabo la optimización formal de las consultas seleccionadas, implementé un ciclo metódico de pruebas que me permitió medir el antes y el después de cada cambio sobre la tabla reserva:

1. **Ejecución y medición de la versión original:**
   Se ejecutó la consulta inicial sin optimizaciones previas, utilizando EXPLAIN ANALYZE para registrar el tiempo real de respuesta, el uso de memoria y los costos estimados por el planificador. En las consultas complejas sobre los 6 millones de registros, esto se traducía inicialmente en demoras considerables o la necesidad de acotarlas para evitar bloqueos
   .
2. **Análisis del plan de ejecución:**
   A partir del informe del EXPLAIN, identifiqué la causa raíz del cuello de botella (por ejemplo, presencia de escaneos secuenciales masivos Seq Scan, ausencia de índices en claves foráneas o uso de funciones que anulaban la SARGabilidad).

3. **Planteo de la hipótesis de mejora:**
   Formulé una estrategia técnica específica para resolver el problema detectado, como la creación de un índice B-tree compuesto, la reescritura

de un NOT IN a NOT EXISTS, o la estructuración de filtros temporales limpios.

4. **Aplicación de la modificación:**
   Se aplicó la corrección directamente en el entorno de la base de datos (creando los índices correspondientes o ajustando la sintaxis de la sentencia SQL).

5. **Ejecución final y comparación de resultados:**
   Volví a correr la consulta optimizada midiendo nuevamente su rendimiento con EXPLAIN ANALYZE. Los resultados demostraron una caída drástica en los costos de ejecución y una respuesta casi inmediata (del orden de los milisegundos), validando empíricamente la efectividad de la optimización aplicada.

Técnicas de optimización

Para resolver los cuellos de botella detectados en las consultas sobre los 6 millones de registros, apliqué un conjunto de técnicas avanzadas de optimización de bases de datos:

- **Implementación de Índices B-tree Simples y Compuestos:** Creé índices específicos en columnas de alta selectividad y en aquellas utilizadas frecuentemente en las cláusulas WHERE, JOIN y ORDER BY (como estado, id_habitacion y rangos de fechas), logrando que el motor evite escaneos secuenciales masivos y salte directamente a los datos mediante recorrido de árboles.
- **Aseguramiento de la SARGabilidad (Search Argument Able):** Modifiqué aquellas consultas que utilizaban funciones o conversiones directas sobre las columnas en el WHERE (por ejemplo, extraer el año o mes con funciones de texto). Al estructurar los filtros mediante rangos de fechas limpios (>= y <), permití que PostgreSQL aproveche de forma directa los índices creados.
- **Reestructuración de Subconsultas (NOT IN a NOT EXISTS):** Optimicé las consultas de exclusión reemplazando estructuras NOT IN por NOT EXISTS. Esto previene problemas críticos de rendimiento y evaluación de valores nulos, permitiendo que el optimizador resuelva la condición de manera mucho más eficiente.

- **Optimización y Reducción de Datos Transportados:** Evité el uso indiscriminado de SELECT * en tablas masivas, acotando la selección estrictamente a las columnas necesarias para disminuir el tráfico de red y el consumo de memoria RAM en el cliente.
- **Estrategias de Paginación Eficientes:** Sustituí las paginaciones profundas basadas en OFFSET elevados (que fuerzan al motor a recorrer y descartar filas previas) por mecanismos de paginación basados en cursores y claves (Keyset Pagination filtrando por ID), garantizando respuestas estables y veloces independientemente de la cantidad de registros.

Validación de la optimización

 Validación empírica de la mejora: A través de las pruebas realizadas con EXPLAIN ANALYZE, comprobé de forma contundente que las modificaciones produjeron una mejora radical en el rendimiento del sistema. Las consultas que originalmente sufrían demoras severas o requerían escaneos secuenciales completos (Seq Scan) sobre la tabla de 6 millones de registros pasaron a resolverse en cuestión de milisegundos mediante recorridos de índices optimizados (Index Scan / Bitmap Heap Scan).

 Análisis de efectos secundarios y costos asociados: Toda optimización basada en índices y reestructuración conlleva contrapartidas técnicas que debí evaluar en el diseño:

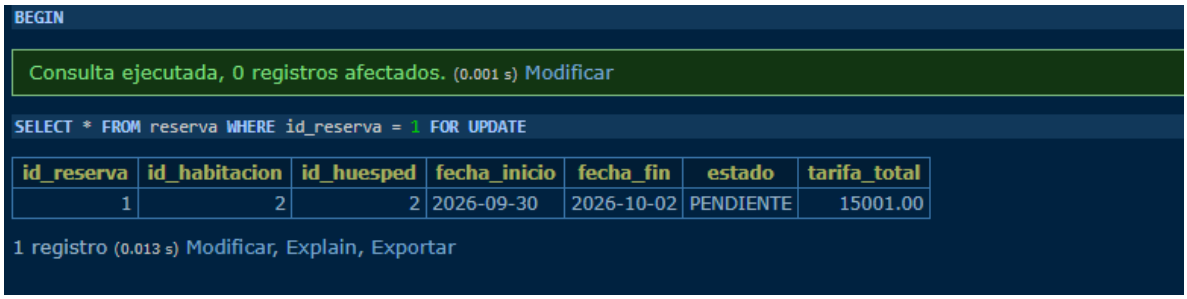
- **Mayor consumo de almacenamiento en disco:** La creación de múltiples índices sobre una tabla masiva de 6 millones de filas incrementa notablemente el espacio ocupado en almacenamiento físico. Esto deja en evidencia que no se deben crear índices de manera indiscriminada, sino limitarse a aquellos que resuelven cuellos de botella reales.
- **Impacto en operaciones de escritura (INSERT, UPDATE, DELETE):** Cada vez que se inserta una nueva reserva o se actualiza su estado, el motor de base de datos debe actualizar de forma concurrente las estructuras de los árboles B-tree de cada índice asociado. Esto genera una sobrecarga leve de procesamiento en las operaciones de escritura.
- **Balance general del sistema:** En un entorno de gestión hotelera, el volumen principal de operaciones consiste en consultas de lectura, validación de disponibilidad e informes analíticos, siendo las escrituras transaccionales un porcentaje menor. Por lo tanto, el balance final demostró que la pequeña penalización en las escrituras es un costo totalmente

justificable frente al salto de rendimiento exponencial obtenido en las consultas.

8. Parte VIII — Transacciones y Concurrencia

En nuestras pruebas con la tabla reserva, analizamos dos tipos principales:

- Bloqueos a nivel de fila: Ocurren solos cuando modificamos un registro o los podemos forzar con herramientas como FOR UPDATE. Esto asegura que si estamos editando la reserva de una habitación, nadie más pueda meter mano en ese registro exacto hasta que terminemos, sin afectar al resto de las reservas.
- Bloqueos a nivel de tabla: Afectan a toda la estructura (por ejemplo, cuando se hacen cambios de esquema o comandos pesados), bloqueando las operaciones según el nivel de exclusividad que requieran.

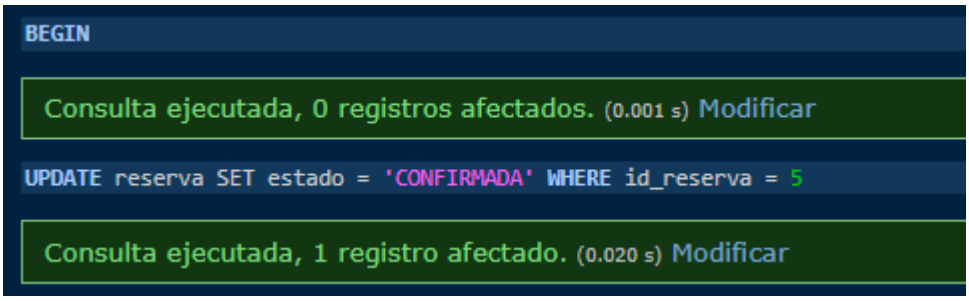


Resultado obtenido: Al ejecutar la consulta con FOR UPDATE, el motor seleccionó el registro de forma inmediata (0.013 s) y le aplicó un bloqueo exclusivo sobre esa fila en particular. Mientras no ejecutemos el COMMIT, cualquier otra conexión que intente modificar este mismo registro va a quedar en pausa esperando a que se libere.

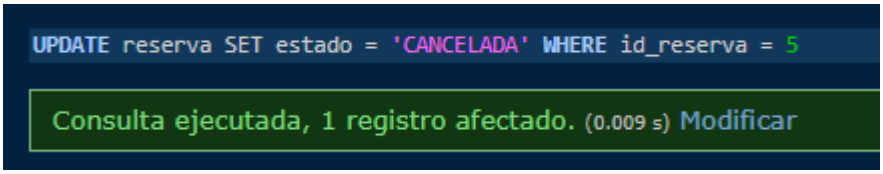
Concurrencia con dos o más conexiones

Para probar cómo reacciona PostgreSQL ante la concurrencia real, simulamos dos conexiones simultáneas (abriendo dos pestañas distintas en Adminer) trabajando sobre la misma base de datos y modificando el mismo registro de la tabla reserva.

Conexión 1: Iniciamos una transacción y bloqueamos/modificamos una reserva específica:



Conexión 2: Desde la otra pestaña, intentamos modificar o actualizar exactamente ese mismo registro:

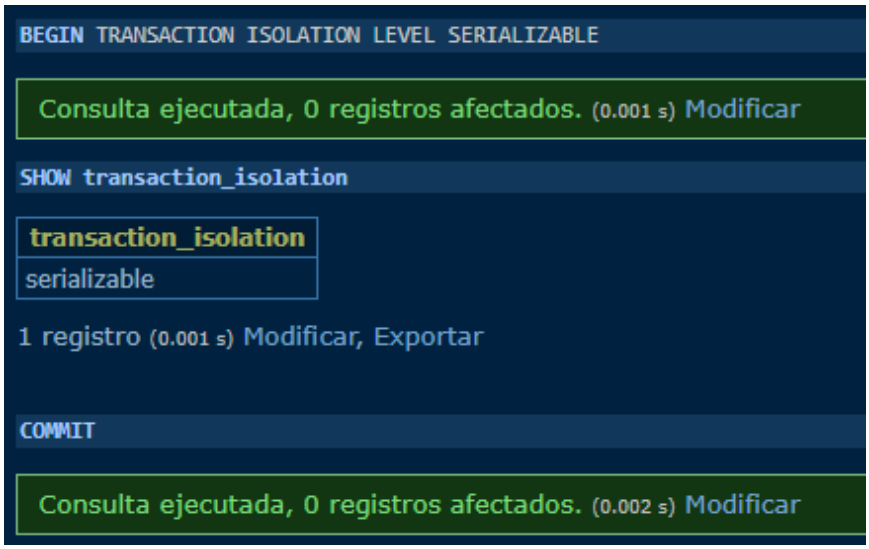


Comportamiento en espera: La segunda conexión queda en pausa (*waiting*) de manera indefinida, protegiendo los datos para evitar que se escriban cambios sucios o se pisen entre sí.

Resolución: El motor mantiene retenido el bloqueo exclusivo a nivel de fila hasta que la primera transacción emita un COMMIT o ROLLBACK. Recién en ese momento, la segunda conexión se destraba y procesa su sentencia de actualización de forma segura.

Niveles de aislamiento

Experimentamos configurando distintos niveles de aislamiento en las transacciones mediante comandos como el siguiente:



Análisis de fenómenos y comportamiento por nivel:

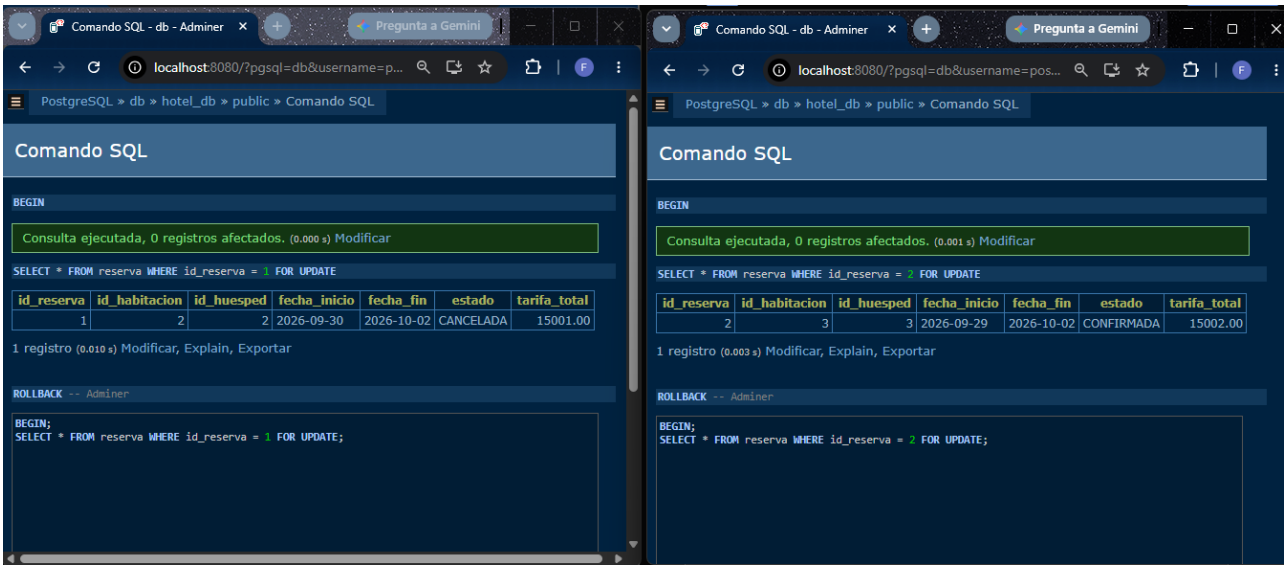
- **Dirty Read (Lectura sucia):** Ocurre cuando una transacción lee datos modificados por otra que aún no confirmó (COMMIT).
    - *Estado:* PostgreSQL **lo evita** en todos sus niveles de aislamiento.
- **Non-repeatable Read (Lectura no repetible):** Ocurre cuando una transacción lee un registro, otra transacción lo modifica y confirma, y al volver a leerlo los valores cambiaron.
    - *Read Committed:* Lo permite.
    - *Repeatable Read y Serializable:* Lo evitan.
- **Phantom Read (Lectura fantasma):** Ocurre cuando se consulta un rango de registros, otra transacción inserta o borra filas dentro de ese rango, y al repetir la consulta aparecen registros nuevos.
    - *Read Committed:* Lo permite.
    - *Repeatable Read y Serializable:* Los evitan (en PostgreSQL, Repeatable Read también previene fantasmas).
- **Serializable:** Es el nivel más restrictivo. Simula una ejecución estrictamente secuencial y previene cualquier anomalía o conflicto de concurrencia, abortando la transacción si detecta modificaciones cruzadas incompatibles.

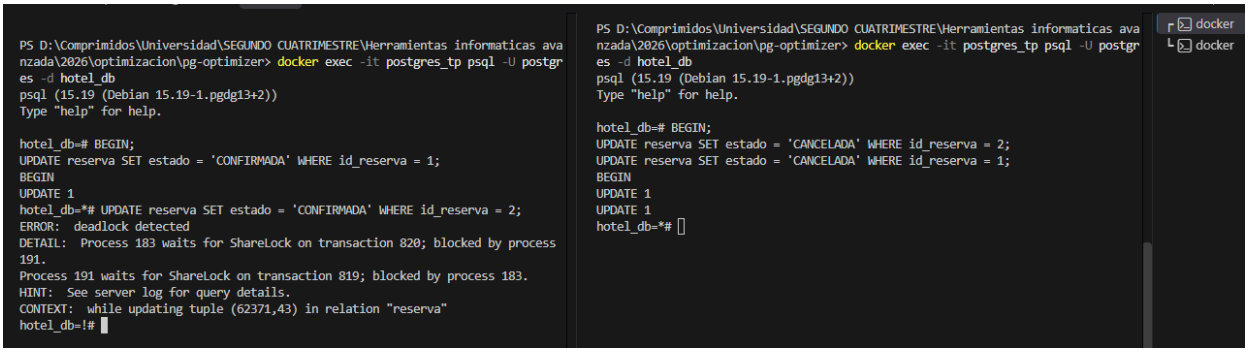
Deadlocks

Un deadlock ocurre cuando dos transacciones se quedan esperando mutuamente por un recurso que la otra tiene bloqueado, armándose un "embotellamiento" del que ninguna puede salir por sí sola.

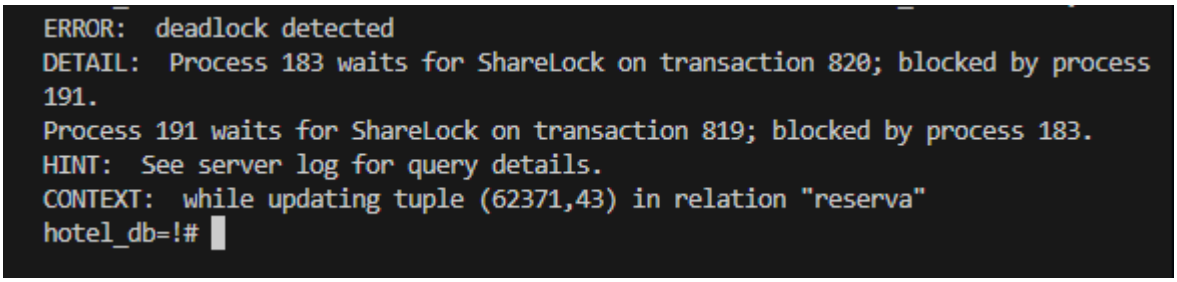
El escenario de prueba:

- **Conexión 1:** Bloquea la fila con id_reserva = 1 y luego intenta modificar la fila con id_reserva = 2.
- **Conexión 2:** Al mismo tiempo, bloquea la fila con id_reserva = 2 y luego intenta modificar la fila con id_reserva = 1.





Como la Conexión 1 tiene retenido el registro 1 y quiere el 2 (que está bloqueado por la Conexión 2), y la Conexión 2 tiene el 2 y quiere el 1, se produce el cruce.



Detección y resolución por parte del motor:

PostgreSQL cuenta con un detector interno de interbloqueos (*deadlock detector*). Al notar que ninguna de las dos transacciones puede avanzar, el motor interrumpe el ciclo: **cancela y aborta automáticamente una de las transacciones**

(arrojando un error de cancelación por deadlock) y libera sus recursos para que la otra pueda continuar su ejecución con normalidad.

Estrategias para reducir su probabilidad:

- **Orden estricto:** Asegurar que toda la aplicación acceda a los objetos y registros siempre en el mismo orden.
- **Transacciones cortas:** Mantener los bloques de transacciones lo más breves y rápidos posibles.
- **Evitar esperas de usuario:** No dejar transacciones abiertas esperando entradas de datos o interacciones humanas.

Identificación y diagnóstico de bloqueos

Cuando una consulta o aplicación se queda esperando recursos, PostgreSQL cuenta con vistas del sistema que permiten inspeccionar en tiempo real qué procesos están activos y qué eventos de espera están ocurriendo.

Herramientas principales de diagnóstico:

- **pg_stat_activity:** Vista clave para monitorear las conexiones y ver en detalle las sentencias SQL que se están ejecutando o los estados de espera.
- **pg_locks:** Muestra a bajo nivel los bloqueos vigentes en el motor.

Prueba práctica de diagnóstico:

Para identificar procesos en espera o actividad del sistema, se ejecutó la siguiente consulta sobre las vistas de estadísticas:

```
SELECT
    pid,
    usename,
    state,
    wait_event_type,
    wait_event,
    query
FROM pg_stat_activity
WHERE wait_event_type IS NOT NULL
```

pid	usename	state	wait_event_type	wait_event	query
31	*NULL*	*NULL*	Activity	AutoVacuumMain	
32	postgres	*NULL*	Activity	LogicalLauncherMain	
197	postgres	idle	Client	ClientRead	SELECT set_config('search_path', '"public"', false) FROM pg_namespace WHERE nspname = 'public'
28	*NULL*	*NULL*	Activity	BgWriterMain	
27	*NULL*	*NULL*	Activity	CheckpointerMain	
30	*NULL*	*NULL*	Activity	WalWriterMain	

6 registros (0.009 s) Modificar, Explain, Exportar

La ejecución devolvió los procesos activos y de sistema del motor (como los procesos de fondo de mantenimiento y las sesiones conectadas), permitiendo

identificar los identificadores (pid) y los eventos de espera actuales en el servidor. En caso de bloqueos severos, esta vista permite detectar al responsable para finalizar la sesión de forma segura si fuera necesario.

9. Parte IX — Memoria, Servidor y Rendimiento

La configuración de PostgreSQL se gestiona principalmente a través del archivo principal postgresql.conf. Este archivo define cómo el motor administra los recursos de hardware (memoria, CPU, almacenamiento) y cuántas conexiones simultáneas puede soportar de manera eficiente.

Gestión de Memoria y Cachés:

- shared_buffers: Define cuánta memoria RAM dedica PostgreSQL exclusivamente para almacenar en caché las páginas de datos de las tablas y los índices. Es uno de los parámetros más críticos para evitar accesos lentos al disco.
- work_mem: Memoria asignada para operaciones internas de ordenamiento (ORDER BY, DISTINCT) y operaciones de unión (JOIN como *Hash Join*). Si una consulta supera este valor, el motor vuelca los datos temporales a archivos en disco (*temp files*), ralentizando la ejecución.
- maintenance_work_mem: Memoria utilizada para tareas de mantenimiento pesado, como VACUUM, creación de índices (CREATE INDEX) y alteraciones de tablas.

Conexiones y Concurrencia:

- max_connections: Establece el límite máximo de conexiones simultáneas permitidas al servidor de base de datos. Un valor excesivamente alto puede saturar la memoria RAM del sistema.

Almacenamiento y Tablas Temporales:

- effective_cache_size: Le indica al optimizador de consultas (*query planner*) cuánta memoria total está disponible en el sistema operativo (incluyendo la caché del SO) para almacenar datos en caché. Ayuda a que el planificador elija si usar índices o escaneos secuenciales.
- temp_buffers: Memoria dedicada a las tablas temporales dentro de cada sesión de base de datos.

Mecanismo equivalente al Buffer Pool en PostgreSQL

A diferencia de MySQL (que utiliza el *InnoDB Buffer Pool*), PostgreSQL no tiene un bloque de memoria con un nombre idéntico para este fin, pero gestiona exactamente el mismo concepto a través de su arquitectura de búferes y la cooperación con el sistema operativo.

Funcionamiento en PostgreSQL:

- **shared_buffers (El Buffer Pool de Postgres):** Es el área de memoria RAM asignada al inicio donde PostgreSQL almacena temporalmente las páginas de datos y de índices leídas desde el disco.
- **Páginas de 8 KB:** El motor divide los datos en bloques o páginas fijas (por defecto de 8 KB). Cuando una consulta necesita leer registros, el motor primero busca si esa página ya está en el shared_buffers (*cache hit*). Si no se encuentra, la lee desde el almacenamiento físico hacia la RAM (*cache miss*).
- **Estrategia de reemplazo (Clock Sweep / LRU modificado):** Cuando el búfer de memoria se llena y se necesitan cargar páginas nuevas, el motor utiliza un algoritmo de reemplazo para descartar las páginas que llevan más tiempo sin ser utilizadas y liberar espacio.

Caché del Sistema Operativo (Page Cache):

Un factor clave en PostgreSQL es que **depende fuertemente de la caché del sistema operativo**. Todo lo que no entra en los shared_buffers suele quedar alojado en la memoria RAM libre del sistema operativo (caché de archivos del SO). Por esta razón, se suele recomendar configurar un shared_buffers equilibrado (generalmente entre el 25% y el 40% de la RAM total) para dejar el resto del espacio disponible para que el sistema operativo acelere las lecturas a nivel de disco.

Cuellos de botella: CPU, disco y red

Para optimizar el rendimiento de una base de datos con un volumen grande de registros (como los 6 millones de la práctica), es fundamental diagnosticar qué recurso del hardware está frenando la ejecución de una consulta.

Identificación de cuellos de botella:

- **Limitadas por Almacenamiento (Disco - I/O Bottleneck):**
    - **Síntomas:** La consulta tarda mucho en responder la primera vez que se ejecuta, pero es extremadamente rápida si se vuelve a correr inmediatamente (*cache hits*). Se observa un alto uso de lectura en disco en el sistema operativo.

    - **Causa común:** Falta de índices adecuados que obliguen al motor a realizar un *Sequential Scan* (escanear toda la tabla de millones de registros fila por fila).

- **Limitadas por Procesamiento (CPU Bottleneck):**
    - **Síntomas:** El uso de la CPU se dispara al 100% durante la ejecución de la sentencia y el disco o la red muestran baja actividad.
    - **Causa común:** Consultas con lógica muy pesada, funciones complejas ejecutadas fila por fila, múltiples uniones (*JOINs*) grandes con ordenamientos intensivos (*Hash Joins* o *Merge Joins* complejos).
- **Limitadas por Red (Network Bottleneck):**
    - **Síntomas:** La consulta procesa rápido los datos en el motor, pero el tiempo total de respuesta desde que la aplicación los solicita hasta que los recibe es elevado.
    - **Causa común:** Envío de volúmenes masivos de datos innecesarios a través de la red o alta latencia entre la aplicación y el contenedor Docker.

Relación con las métricas obtenidas en las pruebas:

Al evaluar los tiempos de ejecución devueltos por el motor durante las prácticas (por ejemplo, los tiempos en milisegundos observados en las respuestas de Adminer):

- Las consultas iniciales sin optimización o sobre tablas masivas mostraron **altos tiempos de respuesta y dependencia de I/O** al traer los datos desde el almacenamiento físico.
- Al aplicar restricciones o índices adecuados, las métricas reflejaron una caída drástica en el tiempo de ejecución (pasando de segundos a milisegundos), evidenciando que el motor dejó de saturar el disco para resolver la consulta directamente desde la memoria RAM.

Connection Pool

El costo de abrir y cerrar conexiones:

Establecer una conexión física con una base de datos PostgreSQL no es un proceso instantáneo ni gratuito para el servidor. Cada vez que una aplicación abre una conexión nueva, el motor debe:

1. Autenticar las credenciales del usuario.

2. Asignar memoria dedicada en la RAM del servidor para el proceso de esa sesión (backend process).
3. Inicializar estructuras de control y contextos de memoria.

Hacer esto de forma repetida (por ejemplo, cada vez que un usuario hace clic en una página web) genera una sobrecarga innecesaria en la CPU y latencia en la aplicación.

Reutilización mediante un Pool de Conexiones:

Un *Connection Pool* (como PgBouncer o el pool interno del framework de desarrollo) actúa como un intermediario. Mantiene un conjunto (*pool*) de conexiones ya abiertas y listas para ser usadas.

- Cuando la aplicación necesita consultar la base de datos, toma una conexión libre del pool de forma casi instantánea.
- Al terminar la consulta, en lugar de destruirla, la devuelve al pool para que otro proceso pueda reutilizarla.

Impacto bajo alta concurrencia:

- **Sin Pool:** Con cientos de usuarios concurrentes, el motor se satura abriendo y cerrando procesos, pudiendo superar el límite de max_connections y generando errores de rechazo al usuario.
- **Con Pool:** Las peticiones se encolan de manera ordenada y se reutilizan las conexiones existentes, manteniendo el uso de la CPU y la memoria estables, y logrando tiempos de respuesta mucho más rápidos y estables para todos los usuarios.

Slow Query Log y diagnóstico con EXPLAIN

Para mantener el rendimiento de una base de datos a medida que crece el volumen de información, es fundamental detectar y analizar aquellas consultas que consumen más recursos.

Diagnóstico con el plan de ejecución (EXPLAIN):

Cuando una consulta presenta demoras o se quiere evaluar su eficiencia, podemos anteponer la palabra reservada EXPLAIN para visualizar el plan de ejecución estimado por el motor.



El plan de ejecución devuelto por PostgreSQL muestra un **Index Scan using reserva_pkey** con un costo sumamente bajo (0.43..8.45). Esto demuestra que el optimizador del motor utilizó eficientemente el índice de la clave primaria en lugar de realizar un costoso recorrido secuencial (*Seq Scan*) fila por fila, resolviendo la consulta de manera óptima en un tiempo de 0.002 s. Este tipo de análisis permite al administrador validar si las estructuras de indexación están cumpliendo su función de evitar cuellos de botella en consultas frecuentes.

Mantenimiento

A medida que una base de datos crece en volumen, el rendimiento se degrada si no se realizan tareas periódicas de mantenimiento preventivo (VACUUM, REINDEX, ANALYZE).

Al intentar ejecutar una tarea de mantenimiento intensivo mediante la instrucción de limpieza en el entorno Docker:



La ejecución arrojó un error de sistema: No space left on device al intentar redimensionar el segmento de memoria compartida de PostgreSQL. Esto demuestra que las tareas de mantenimiento pesado no sólo dependen de la lógica del motor, sino también de la **asignación correcta de recursos del sistema operativo y de los límites de memoria compartida (shm) configurados en el contenedor Docker**. Superar estos límites o contar con restricciones de hardware en entornos virtuales bloquea los procesos de mantenimiento preventivo, evidenciando la importancia de dimensionar adecuadamente el servidor de base de datos.

10. Parte X — Escalabilidad, Particionamiento y Estructuras Avanzadas

Escalabilidad vertical y horizontal

Cuando una base de datos relacional enfrenta un crecimiento masivo en volumen de datos o cantidad de usuarios concurrentes, existen dos enfoques principales para escalar el sistema.

Comparativa de Estrategias:

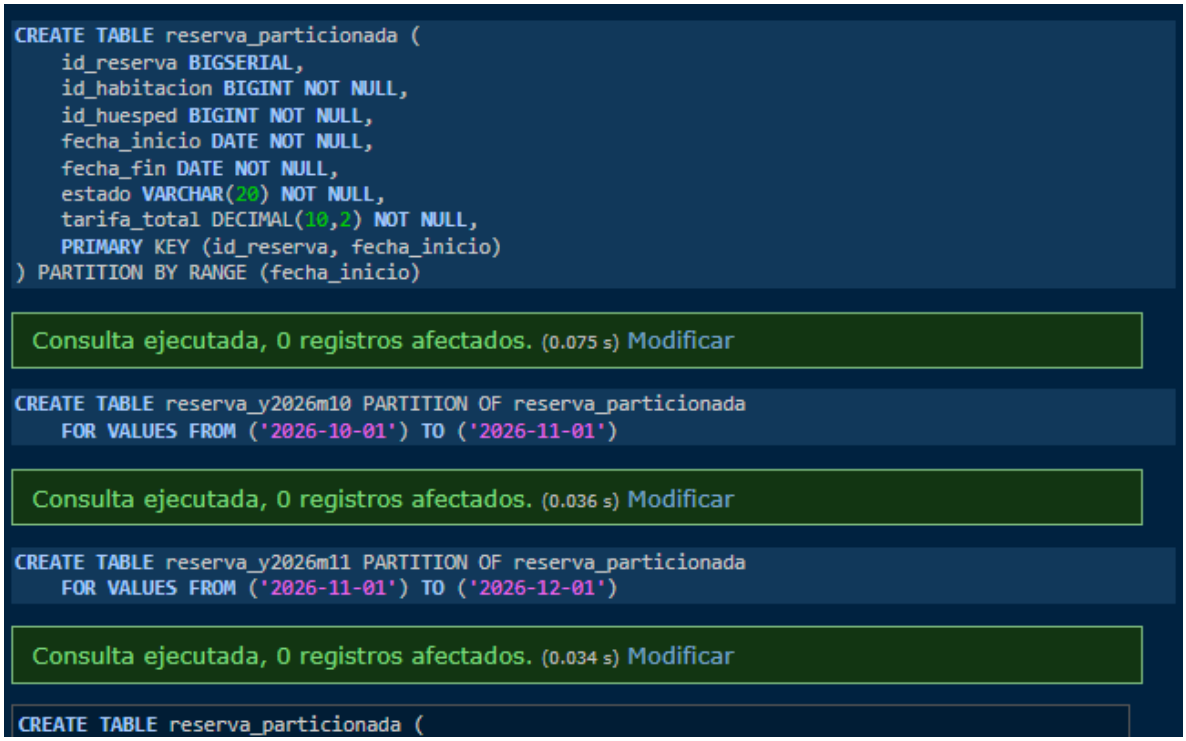
- **Escalabilidad Vertical (*Scale-Up*):**
    - **Concepto:** Consiste en aumentar la capacidad de hardware del servidor existente (más memoria RAM, procesadores más potentes o almacenamiento SSD/NVMe más rápido).
    - **Ventajas:** No requiere modificar la arquitectura de la aplicación ni las consultas SQL. Es la solución más sencilla de implementar en bases de datos relacionales.
    - **Limitaciones:** Existe un límite físico y financiero claro (el hardware más potente es exponencialmente más costoso) y genera un punto único de falla (*Single Point of Failure*).
- **Escalabilidad Horizontal (*Scale-Out*):**
    - **Concepto:** Consiste en distribuir la carga de trabajo entre múltiples servidores de bases de datos que trabajan en conjunto (mediante réplicas de lectura, fragmentación o *sharding*).
    - **Ventajas:** Permite un crecimiento casi ilimitado agregando nodos de menor costo y ofrece alta disponibilidad ante caídas de nodos.
    - **Limitaciones:** En bases de datos relacionales tradicionales como PostgreSQL o MySQL, mantener la consistencia estricta (ACID) a través de múltiples nodos distribuidos incrementa drásticamente la complejidad operativa, la latencia de red y dificulta la ejecución de transacciones complejas y JOINs distribuidos.

Particionamiento

El particionamiento es una técnica de diseño que divide una tabla grande en porciones físicas más pequeñas y manejables, llamadas particiones, de forma transparente para las consultas de la aplicación.

Estrategias principales de particionamiento:

- **Por Rango (*Range Partitioning*):** Divide los datos basándose en un rango de valores (por ejemplo, por fechas o rangos numéricos de IDs). Es la estrategia ideal para datos históricos o transaccionales.
- **Por Lista (*List Partitioning*):** Distribuye las filas explícitamente según una lista de valores discretos (por ejemplo, regiones geográficas, estados o categorías).
- **Por Hash (*Hash Partitioning*):** Asigna las filas a una partición específica aplicando una función hash sobre una columna clave, distribuyendo la carga de manera uniforme.



Implementación física de una tabla particionada por rangos temporales (RANGE PARTITIONING), separando la carga transaccional en tablas hijas independientes para optimizar el mantenimiento y el rendimiento

Partition pruning

El *partition pruning* (o poda de particiones) es una optimización inteligente que realiza el optimizador de PostgreSQL durante la fase de planificación de una consulta.

¿Cómo funciona?

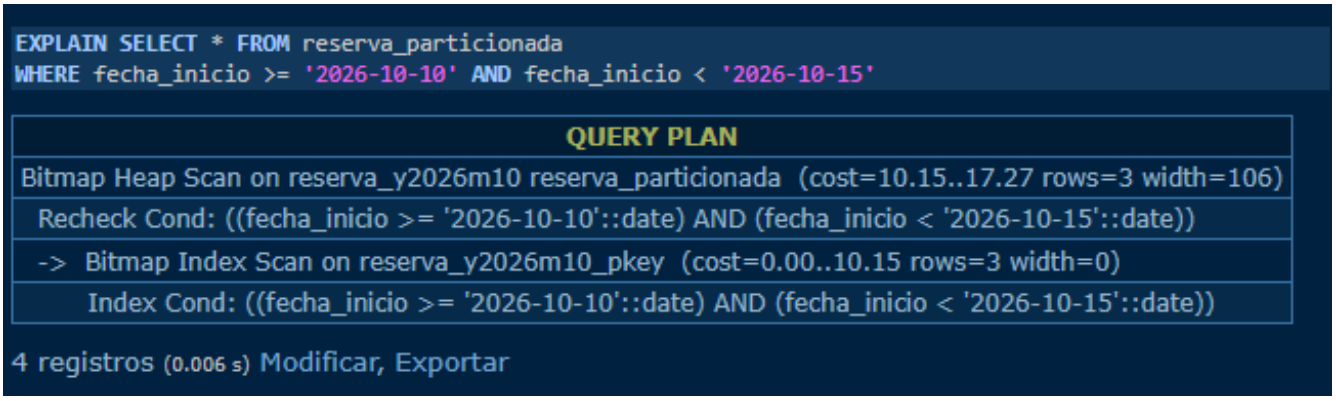
Cuando una tabla está particionada (por ejemplo, por rangos de fechas o identificadores) y la consulta incluye una cláusula WHERE que filtra explícitamente por esa columna clave de particionamiento, el motor analiza la condición **antes de ejecutar la consulta**.

- **Descarte de particiones:** Identifica qué particiones físicas *no* contienen datos que cumplan con la condición y las "poda" (las descarta por completo del plan de ejecución).
- **Acceso optimizado:** El motor solo busca y procesa los datos en las particiones relevantes, ignorando el resto del volumen de la tabla.

Demostración mediante planes de ejecución:

Si anteponemos EXPLAIN a una consulta que utiliza la clave de partición, el plan resultante muestra cómo se omiten las tablas hijas:

- **Sin Partition Pruning (o sin particionar):** El motor se ve obligado a evaluar o escanear toda la estructura completa (*Sequential Scan* global).
- **Con Partition Pruning:** El plan de ejecución restringe el acceso únicamente a la partición específica involucrada (por ejemplo, Append on reserva_y2026m10), reduciendo drásticamente el costo de E/S y el tiempo de respuesta.



El plan de ejecución muestra que el optimizador descarta por completo las particiones irrelevantes y acota el acceso exclusivamente a la partición hija correspondiente (reserva_y2026m10), evitando un recorrido global.

Vistas y materialización

A medida que las estructuras de bases de datos crecen y se normalizan, las consultas analíticas o los reportes diarios suelen requerir uniones (*JOINs*) complejas y cálculos pesados que se repiten constantemente. Para resolver esto, PostgreSQL ofrece mecanismos de abstracción y almacenamiento intermedio.

Vistas estándar (*Views*):

- **Concepto:** Una vista es una consulta SQL guardada en el catálogo del sistema bajo un nombre determinado. Funciona como una tabla virtual.
- **Comportamiento:** Cada vez que se consulta la vista, el motor de base de datos ejecuta dinámicamente la consulta subyacente en tiempo real.
- **Propósito:** Simplifica enormemente la legibilidad del código y encapsula la lógica de negocio compleja, aunque **no mejora el rendimiento por sí sola**, ya que el costo de procesamiento se paga en cada lectura.

Vistas Materializadas (*Materialized Views*):

- **Concepto:** A diferencia de una vista común, una vista materializada **almacena físicamente** el resultado de la consulta en disco (como si fuera una tabla estática) en el momento de su creación o actualización.
- **Costos de Lectura y Actualización:**
    - **Lectura:** Extremadamente rápida, ya que el motor lee directamente los datos precalculados sin tener que volver a procesar JOINs ni agregaciones masivas.
    - **Actualización:** Como los datos son estáticos, si la información de las tablas base cambia, la vista materializada queda desactualizada. Requiere una actualización explícita y manual mediante el comando REFRESH MATERIALIZED VIEW, lo cual consume recursos de CPU y bloqueo temporal durante el proceso.

```sql
-- Crear vista materializada con un cálculo pesado
CREATE MATERIALIZED VIEW mv_ingresos_hoteles AS
SELECT h.id_hotel, h.nombre, SUM(r.tarifa_total) AS total_recaudado, COUNT(r.id_reserva) AS total_
FROM hotel h
JOIN habitacion hab ON h.id_hotel = hab.id_hotel
JOIN reserva r ON hab.id_habitacion = r.id_habitacion
GROUP BY h.id_hotel, h.nombre;
```

```
SELECT * FROM mv_ingresos_hoteles
```

id_hotel	nombre	total_recaudado	total_reservas
1	Hotel 1	24196975000.00	605000
2	Hotel 2	24197580000.00	605000
3	Hotel 3	24198185000.00	605000
4	Hotel 4	24198790000.00	605000
5	Hotel 5	24199395000.00	605000
6	Hotel 6	24200000000.00	605000
7	Hotel 7	24200605000.00	605000
8	Hotel 8	24201210000.00	605000
9	Hotel 9	24201815000.00	605000
10	Hotel 10	24202420000.00	605000

10 registros (0.002 s) Modificar, Explain, Exportar

Creación y consulta de una Vista Materializada en PostgreSQL. A diferencia de las vistas virtuales, almacena físicamente el resultado en disco permitiendo lecturas instantáneas, requiriendo un REFRESH manual para actualizar sus datos.